Haramaya University

Building the Basis for Development

HARAMAYA UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATICS

DEPARTMENT OF COMPUTER SCIENCE

PROJECT TITLE: ODA BULTUM UNIVERSITY WEBSITE

ORGANIZATION NAME: ODA BULTUM UNIVERSITY

INDUSTRIAL PRACTICE REPORT

Internship Start Date: June 26, 2026

Internship End Date: August 27, 2026

NAME……ID NO

Aregaa Melion…….0739/16

ADVISOR NAME: Mr. Asedo Sh.

SUBMISSION DATE : OCT 12,2026

Declaration

I, Aregaa Melion, hereby confirm that the internship report submitted to the Computer Science department is an honest and accurate reflection of my experience during my internship at OBU University

Name of Student: Aregaa

Signature: _____

Date: _____

Name of Academic Advisor: _____

Signature: _____

Date: _____

Acknowledgements

I would like to express my sincere gratitude to Oda Bultum University for giving me the opportunity to work on this project during my summer internship. My heartfelt thanks go to Mr. Murad, my advisor, for his constant guidance and valuable feedback.

I also thank the ICT Directorate staff for their support and collaboration throughout the project. This experience has helped me connect theory with practice and strengthen my passion for software development

I would like to extend my sincere gratitude to to Mr. Asedo Sh.. for his continuous guidance, encouragement, and insightful feedback throughout my internship project. His support and expertise greatly contributed to the successful completion of my work

List of Abbreviations

3NF – Third Normal Form
ADMIN – Administration / Administrator
API – Application Programming Interface
Auth – Authentication
CMS – Content Management System
CSS – Cascading Style Sheets
DB – Database
HTML – HyperText Markup Language
ICT – Information and Communication Technology
ID – Identification
IT – Information Technology
LAN – Local Area Network
M1–M8 – Milestones 1–8
OBU – Oda Bultum University
OpenAPI – OpenAPI Specification / Open Application Programming Interface
QA – Quality Assurance
RBAC – Role-Based Access Control
RLS – Row-Level Security
TVET – Technical Vocational and Educational Training
UAT – User Acceptance Testing
UI – User Interface
WCAG – Web Content Accessibility Guidelines
WWW – World Wide Web

Table of Contents

CHAPTER 1:
INTRODUCTION 8
1.1 Background of the Organization 8
1.2 Organizational Structure and
Workflow 9
1.3 Team and Department
Overview 10
1.3.1 Daily Workflow and Collaboration
Methods 11
1.4 Project Context 11
1.4.1 Problem Statement 11
1.4.2 Purpose and Goals 11
1.4.3 Scope 11

1.5 Objectives 11
1.5.1 General Objective 12
1.5.2 Specific Objectives 12
1.6 Organization of the Document 13

CHAPTER 2: SOFTWARE DEVELOPMENT PLAN 12
2.1 Project Schedule 12
2.1.1 Project Activities and Timeline 13
2.1.2 Project Milestones 14
2.1.3 Gantt-Style Schedule 15
2.2 Resources 15
2.2.1 Human Resources 15
2.2.2 Hardware Resources 16
2.2.3 Software Resources 17
2.2.4 Database Resources 17
2.2.5 Authentication and Security Resources 18
2.2.6 Development and Testing Resources 18
2.2.7 Communication and Collaboration Resourc 18
2.3 Risk Management 19
2.3.1 Risk Identification 19
2.3.2 Technical Risk Management 20
2.3.3 Authentication and Security Risk Management 20
2.3.4 Testing and Debugging as Risk 21
2.3.5 Risk Monitoring and Response 21
2.3.6 Risk Management Summary 22
2.4 Development Methodology 22
2.4.1 Development Approach 23
2.4.2 Application of the Methodology 24
2.4.3 Database and Security Integration 25
2.4.4 Why the Iterative Approach Was Appropriate 25
2.5 Quality Assurance 26
2.5.1 Quality Assurance Objectives 26
2.5.2 Testing Activities 26
2.5.3 Frontend Quality Assurance 27
2.5.4 Backend and Database Quality Assurance 28
2.5.5 Authentication and Security Quality Assurance 28
2.5.6 Code Quality and Development Checks 29

2.5.7 Documentation and Review 29
2.5.8 Quality Assurance Process 29
2.5.9 Quality Assurance Summary 29
2.6 Required Project Documents 30

CHAPTER 3: SYSTEM REQUIREMENTS 30
3.1 Existing System Analysis 30
3.1.1 Overview of the Existing System 30
3.1.2 Problems Identified 30
3.1.3 Existing System Limitations 31
3.1.4 Need for Improvement 31
3.2 Proposed System 31
3.2.1 Main Features 31
3.2.2 System Architecture 32
3.2.3 Administrative CMS 32
3.2.4 Security 32
3.2.5 Expected Benefits 32
3.3 Functional Requirements 33
Table 3.2: Summary of Functional Requirements 34
3.4 Non-Functional Requirements 35
Table 3.3: Non-Functional Requirements 35

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN 36
4.1 System Analysis and Design Overview 36
4.2 System Models 37
4.2.1 System Actors 37
4.2.2 Use Case Model 37
4.2.3 Activity Model 38
4.2.4 Sequence Model 39
4.2.5 Class/Object Model 40
4.2.6 Model Integration 41
4.3 User Interface Design and NavigationalFlow 41
4.3.1 Public Website Interface 42
4.3.2 Navigation Design 43

4.3.3 News, Events, and Media 43
4.3.4 Administrative CMS Interface 44
4.3.5 Forms and Media Management 45
4.3.6 Authentication and Responsive Desig..... 45
4.4 Software Architecture and Data Management Design 46
4.4.1 System Architecture 46
4.4.2 Data Management 47
4.4.3 Data Flow 48
4.4.4 Authentication and Security 48
4.4.5 Media Management 48
4.5 Security and Access Control Design 48
4.5.1 Authentication and Authorization..... 48
4.5.2 Protected Administrative Access 48
4.5.3 Database Security 48
4.5.4 Session Management 48
4.5.5 Security Testing 49
4.5.6 Security Limitation 49
4.5.7 Security Structure 49
4.6 Deployment and System Infrastructure Design 49
4.6.1 Development Environment 49
4.6.2 System Deployment Structure 49
4.6.3 Database and Services 50
4.6.4 Testing and Version Control 50
4.6.5 Deployment Workflow 50
4.7 Database Design 50
4.7.1 Database Technology 51
4.7.2 Main Data Categories 51
4.7.3 Database and Application Relationship 52
4.7.4 Data Management 52
4.7.5 Database Security 52
4.7.6 Database Verification 52
4.8 Data Flow and Process Design ..... .53
4.8.1 General Data Flow 53
4.8.2 Public User Flow 53
4.8.3 Administrative Flow 53
4.8.4 Content Management Flow 53
4.8.5 Content Retrieval 54

4.8.6 Authentication and Media Flow 54
4.8.7 Security and Verification 54
4.8.8 Overall Data Flow Diagram 54
4.9 System Design Summary 54

CHAPTER FIVE: SYSTEM IMPLEMENTATION 55
5.1 Introduction 56
5.2 Development Environment and Tools 56
5.3 Frontend Implementation 56
5.4 Backend and Database Implementation 56
5.5 Authentication and Security 56
5.6 Media and Content Management 56
5.7 Routing and Responsive Interface 56
5.8 Implementation Challenges and Solutions 56
5.9 Security Findings 57
5.10 Testing 57
5.11 Implementation Results 58

CHAPTER 6: INTERNSHIP REFLECTION AND CONCLUSION 58
6.1 Learning Outcomes 58
6.2 Challenges Faced 58
6.3 Recommendations 58
6.4 Conclusion 58

APPENDICES 59
REFERENCES 61

CHAPTER 1: INTRODUCTION

1. 1 Background of the Organization

Oda Bultum University (OBU) is a public higher education institution established in 2015 under the Ethiopian Ministry of Education, located in Chiro, the capital of West Hararghe Zone,Oromia Regional State. The university was founded to expand access to quality higher education in the eastern regions of Ethiopia, addressing a critical gap in the availability of tertiary institutions serving a large and historically underserved population.

Since its establishment, OBU has grown rapidly in enrollment, academic programs, and physical infrastructure. The university currently serves thousands of undergraduate students across six academic colleges: the College of Agriculture, the College of Natural and Computational Sciences, the College of Business and Economics, the College of Social Sciences and Humanities, the Collegeof Health and Medical Sciences, and the Institute of Technology. These colleges collectively offer a wide range of undergraduate programs tailored to regular, extension, weekend, and summer modalities, serving both fresh school leavers and working professionals seeking degree qualifications.

The university operates under the leadership of a President supported by Vice Presidents responsible for Academic Affairs, Research and Community Service, and Administration and Finance. The Registrar's Office is a central administrative body that operates directly under the Vice President for Academic Affairs and is responsible for the management of all student academic records, enrollment data, grade processing, and transcript issuance across all colleges. As the institution has grown, the demands placed on the Registrar's Office have intensified, exposing significant weaknesses in the existing administrative systems.

The Registrar's Office at OBU serves as the primary internship host organization. During the internship period, the intern worked in close collaboration with the ICT Directorate , which acts as the technical engine of the university, charged with strengthening and expanding digital infrastructure, network systems, and institutional software services and selected academic departments to investigate the systemic inefficiencies of the existing system and to design and develop a replacement system.

Following its development into a university, OBU expanded its academic activities to include several fields of study. Its academic structure includes colleges, institutes, schools, departments, and undergraduate and postgraduate programs.

The university's major institutional activities focus on education, research, technology transfer, and community engagement. It also provides various student, administrative, and digital services to support the university community.

During my apprenticeship, I worked within this institutional environment under the ICT Directorate, where I participated in the development of the university's website.



Figure 1.1: Oda Bultum University Campus

1. 2 Organizational Structure and Workflow

Oda Bultum University operates through a structured organizational system consisting of executive leadership, academic units, administrative directorates, research and community-service units, and support offices.

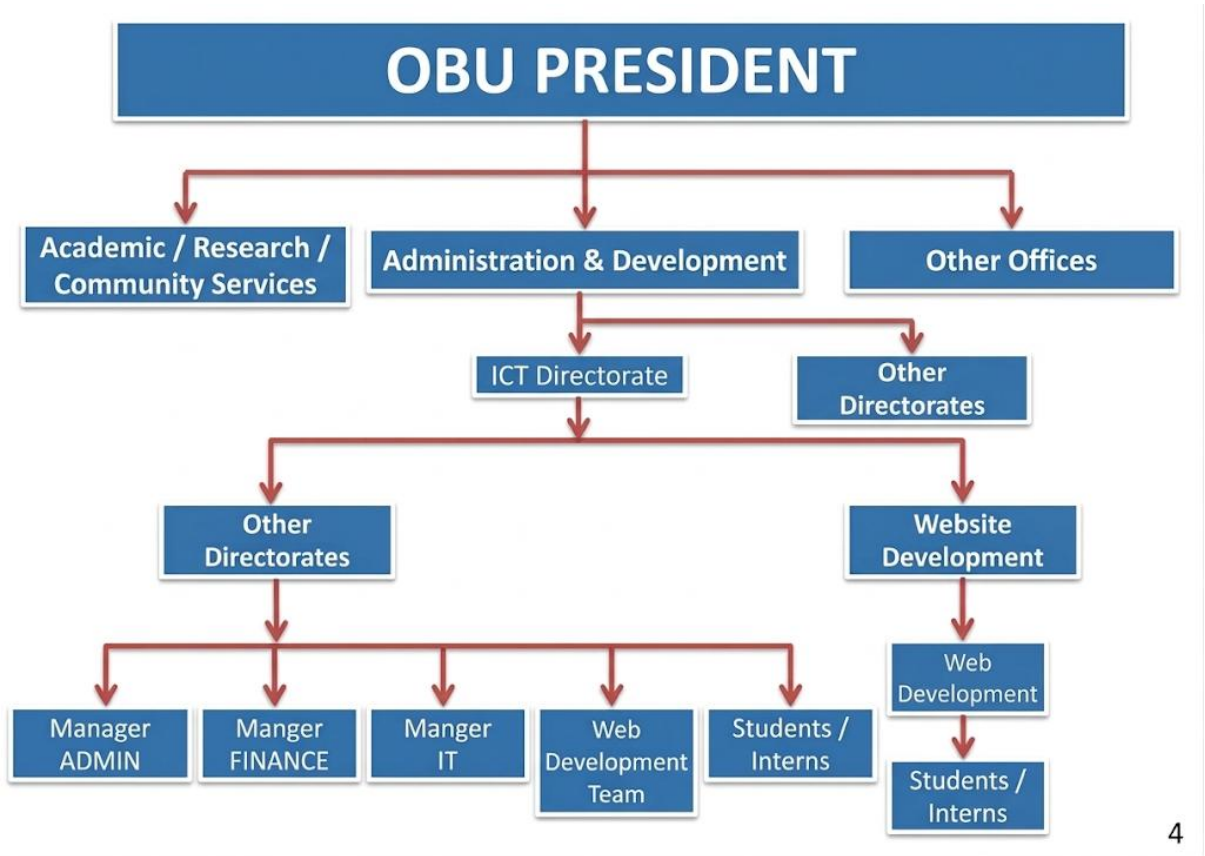
The Office of the President provides overall executive leadership. The President is supported by university bodies and vice-presidential offices responsible for academic, research, technology transfer, community service, administration, and development activities.

The academic structure consists of colleges, institutes, schools, departments, and academic programs. Administrative activities are handled by different directorates and offices, while specialized units support research, community engagement, student services, and other institutional functions.

The ICT Directorate is one of the university offices reporting to the President and is responsible for information and communication technology activities. This directorate was particularly relevant to my apprenticeship because my web development work was conducted within this environment.

Under the guidance of Mr. Murad Abrahim, I worked with other students and interns on the university website. The general project workflow involved identifying requirements, investigating existing functionality, implementing changes, testing the system, correcting problems, and verifying the completed work.

Figure 1.2: Simplified Organizational Structure Relevant to the Apprenticeship



The figure presents a simplified organizational structure relevant to the apprenticeship, showing the position of the ICT Directorate within the university structure and its relationship to the website development activities.

1. 3. Team and Department Overview

The internship was conducted primarily within the ICT Directorate and the Registrar's Office of OdaBultum University. The ICT Directorate serves as the technical backbone of the institution, managing the university's Local Area Network (LAN), server infrastructure, and all deployed software systems. The department is staffed by IT officers, network engineers, and system administrators who are responsible for maintaining uptime, managing user accounts, and providing technical support across all faculties. The intern was assigned to work under the supervision of the ICT Director, collaborating closely with the following personnel:

• ICT Director: Provided institutional guidance, approved the project scope, and coordinated access to server infrastructure and existing system documentation.

• Senior Registrar Officer: Served as the primary domain expert, providing detailed insight into the existing manual workflows, data structures, and pain points in the current system.

• College Assistant Registrars (x3): Representatives from the Colleges of Natural Sciences, Business and Economics, and the Institute of Technology participated in requirements gathering sessions and early UAT walkthroughs.

• Academic Advisor : .provided academic oversight,reviewed technical deliverables, and ensured alignment with the internship program requirements of the Department of Computer Science at Haramaya University. The intern's role was that of a sole full-stack software

developer and systems analyst. Responsibilities included requirement analysis, system architecture design, backend API and frontend development, testing, documentation, and LAN deployment.

1. 3.1. Daily Workflow and Collaboration Methods

Collaboration followed a structured cadence throughout the 45-day internship period. Bi weekly progress meetings were held with the ICT Director every Monday and Thursday to review completed sprint tasks, demonstrate working software increments, and obtain approval for the next sprint's scope. I also conducted ad-hoc requirement validation sessions with the Senior Registrar Officer whenever ambiguity arose around dormitory forms or meal card logic. At the end of each sprint cycle, I submitted written progress summaries to my academic advisor at the end of each sprint cycle. The ICT Directorate's internal operational workflow follows a ticket-based system for user support requests, while infrastructure changes require formal written approval from the ICT Director before deployment. During the internship, all system changes were logged and reviewed with the ICT Director prior to each sprint review, maintaining alignment with the directorate's change

1. 4. Project Context

1. 4.1 Problem Statement

The Registrar's Office and affiliated academic departments at OBU currently operate using a fragmented combination of standalone desktop applications, disconnected web forms, and manual paper-based spreadsheets and also The existing public-facing website was static and did not represent the modern identity of OBU. This created severe operational friction. During peak registration periods, the Registrar's Office and Student Affairs were overwhelmed, resulting in long queues, redundant data entry, lost paperwork, and system bottlenecks. The university urgently needed a unified digital portal that strictly adhered to the OBU Website Governance Policy, which mandates a centralized, secure, and trilingual interface (Afaan Oromoo, English, and Amharic)

- Website information was difficult to update consistently
- Some administrative activities depended on paper-based workflows
- Separate tools could create delays and duplicated work
- Protected content required stronger access control
- Users needed clearer and more accessible online information

1. 4.2 Purpose and Goals

The primary goal of this project was to design and build a unified, accessible website for Oda Bultum University that solves these problems. The new portal combines two key parts into one application: . Public Web Portal: A dynamic, modern front-facing site presenting the university's history, leadership, colleges, research, and campus announcements.

1. 4.3 Scope

The scope covers a complete redesign of the public-facing site and the integrated student self service portal. The system is designed to be hosted on the university's internal LAN and the public web server (www.obu.edu.et) so sensitive student records are protected while public info remains accessible. My responsibilities covered frontend engineering, backend database schema design, responsive layouts, multilingual localization, WCAG 2.1 accessibility features, form validation, testing, deployment, and writing user manuals for both site sections.

1. 5. Objectives

1. 5.1 General Objective

The main objective was to design, develop a responsive university website with an administrative content-management system or function , The system is intended to provide reliable public information and controlled content-management functions for authorized university personnel.

.

1. 5.2 Specific Objectives .

Build a relational database schema normalized to the Third Normal Form (3NF) to prevent data redundancy for student records and administrative data. . Develop a responsive user interface using OBU's official Emerald Green and Gold branding, optimized for mobile, tablet, and desktop use. . Implement Role-Based Access Control (RBAC) to define distinct permissions for System Administrators, Registrar Officers, and Students. Addaccessibility features per WCAG 2.1 standards, including a font-resizer and high-contrast mode, alongside a toggle for three languages (English, Afaan Oromo, Amharic). . Introduce secure authentication and an automated backup mechanism to protect academic records. . Deliver complete technical documentation, including API specs (OpenAPI), an admin manual, and a user guide for the public site

- Gather and document website and content management requirements from relevant stakeholders.
- Design a responsive and usable interface
- Organize website content in a maintainable database structure
- Implement role-based access to administrative functions
- Test core functions, security, and responsive behavior

1.6 Organization of the Document

This report is organized into six chapters. Chapter One introduces the host organization, project context, problem statement, scope, and objectives. Chapter Two presents the software development plan, including schedule, resources, risk management, methodology, quality assurance, and required project documents. Chapter Three describes the existing system, proposed system, and functional and non-functional requirements. Chapter Four explains the system analysis and design, models, user interface, architecture, security, database, and data flow. Chapter Five discusses implementation, tools, challenges, and testing. Chapter Six summarizes learning outcomes, challenges, recommendations, and the conclusion.

CHAPTER 2: SOFTWARE DEVELOPMENT PLAN

2. 1 Project Schedule

The development of the Oda Bultum University (OBU) Official Website and Administrative Content Management System was organized into a sequence of activities and milestones. The schedule was designed to support the development of the public-facing university website together with the protected administrative content management system. The project activities included requirement identification, analysis of the existing system, interface development, backend and database integration, authentication and authorization, content and media management, testing, debugging, security verification, and final review.

The development process followed a feature-by-feature approach. Each major feature was identified, analyzed, implemented, connected to the database where necessary, tested, corrected, and reviewed before moving to the next stage. This approach helped the development team identify problems early and verify the functionality of the system throughout the project

2. 1.1 Project Activities and Timeline

The major activities performed during the project are presented in Table 2.1.

Table 2.1: Project Activities and Schedule

No.	Major Activity	Main Tasks	Expected Deliverable
1	Project Orientation and Requirement Identification	Understand the project objectives, users, major website sections, and administrative requirements	Initial project requirements
2	Existing System Analysis	Inspect the existing project structure, pages, components, database configuration, and current functionality	Existing system analysis
3	System Planning and Design	Organize website sections, navigation, page structure, database-related requirements, and administrative functions	System structure and design plan
4	Frontend Development	Develop and improve pages, reusable components, navigation, layouts, and responsive interfaces	Public website interfaces
5	Backend Development	Develop server-side functionality and connect frontend features with backend operations	Backend functionality
6	Database Integration	Connect the application with PostgreSQL/Supabase and verify data operations	Functional database integration
7	Authentication and Authorization	Implement and test administrator authentication, protected areas, sessions, roles, and access control	Protected administrative system
8	Content Management Development	Implement management of academic information, news, announcements, events, services, research, vacancies, media, and related content	Administrative CMS features
9	Media Management	Implement and test image/media upload and management functionality	Media management functionality
10	Testing and Debugging	Test public pages, administrative functions, database operations, authentication, media, navigation, and responsive behavior	Tested and corrected systemsystemsystem

No.	Major Activity	Main Tasks	Expected Deliverable
11	Security Verification	Verify authentication, authorization, database security, and Row Level Security policies	Security verification results
12	Final Review and Documentation	Review completed features, identify remaining issues, document the work, and prepare the final project report	Final system and documentation

The project used technologies and tools including TypeScript, React, TanStack Start, TanStack Router, Tailwind CSS, Node.js, PostgreSQL, Supabase, Git, GitHub, and testing tools.

2. 1.2 Project Milestones

The main milestones of the project were organized around the completion and verification of major development stages.

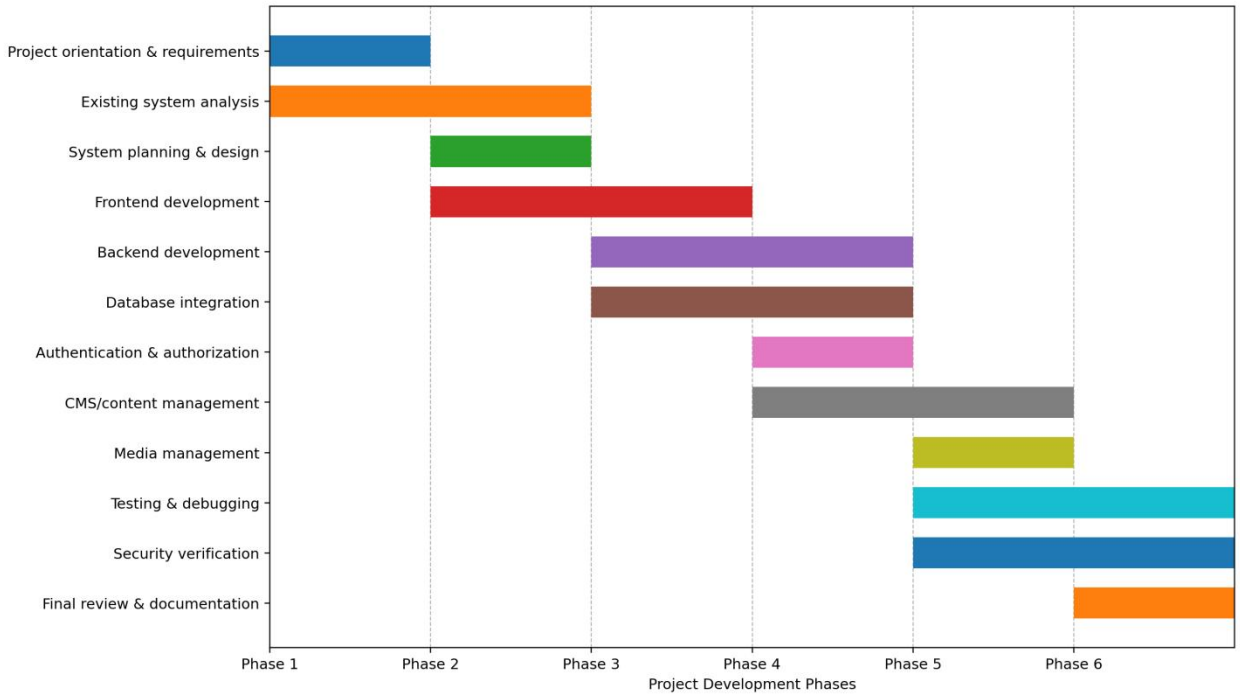
Table 2.2: Project Milestones

Milestone	Description
M1 – Requirements Identified	Major project objectives, users, and functional areas identified
M2 – Existing System Analyzed	Existing application structure and functionality reviewed
M3 – Frontend Structure Completed	Major public pages, layouts, navigation, and reusable components developed
M4 – Backend and Database Integrated	Backend operations connected with PostgreSQL/Supabase
M5 – Administrative CMS Implemented	Major administrative content-management functionality developed
M6 – Authentication and Security Tested	Authentication, authorization, and database security mechanisms verified
M7 – System Testing Completed	Public and administrative functions tested and debugging performed
M8 – Final Review Completed	Completed work reviewed and documented

2. 1.3 Gantt-Style Schedule

A simplified Gantt-style representation of the project schedule is provided below.

Table 2.3: Project Development Schedule (Gantt Chart)



The schedule was iterative rather than strictly linear. Development, testing, debugging, and verification were performed repeatedly as features were implemented. This was consistent with the project's feature-by-feature development process, where implementation was followed by database connection, testing, correction, review, and integration.

The project also involved continuous collaboration and version control using communication channels and Git/GitHub. This supported coordination among team members and helped maintain the project during development.

2. 2 Resources

The development of the Oda Bultum University (OBU) Official Website and Administrative Content Management System required a combination of human resources, hardware resources, software tools, development technologies, database services, and communication and collaboration resources. These resources supported the different stages of the project, including requirements analysis, system development, database integration, testing, debugging, security verification, and documentation.

2. 2.1 Human Resources

The project was carried out within the ICT Directorate of Oda Bultum University. The development work involved collaboration between the intern and the relevant ICT personnel. The intern worked primarily on frontend development and also participated in frontend-backend integration, testing, debugging, and verification activities.

The major human resources involved in the project included:

Web Development Intern: Responsible mainly for developing user interfaces, implementing pages and reusable components, improving navigation and responsive layouts, integrating frontend features with backend functionality, land performing user-facing testing.

ICT/Web Development Personne: Provided project direction, technical guidance, feedback, and assistance during development and testing.

Project Team Members: Collaborated during implementation, troubleshooting, code review, testing, and integration activities.

Academic/Internship Supervisors: Provided academic guidance and monitoring during the industrial practice period.

The project required communication and collaboration throughout development. Team communication was supported through Telegram and in-person discussions, while Git and GitHub were used for source-code collaboration and version control.

2. 2.2 Hardware Resources

The development and testing activities required computer-based resources capable of running the development environment, web browser, database tools, and supporting software.

The main hardware resource was a computer/laptop used for:

Writing and editing source code.

Running the local development environment.

Running and testing the website in web browsers.

Connecting to and verifying the database.

Testing administrative and public website functionality.

Managing project files and documentation.

Using Git and GitHub for version control.

The computer also supported the execution of development and testing tools required by the project.

2. 2.3 Software Resources

Several software tools were used during the development process. The project environment included Visual Studio Code, web browsers, Git, GitHub, Node.js, Supabase, PostgreSQL, Vite, and Playwright/testing tools.

Table 2.4: Software Resources

Software/Tool	Purpose
Visual Studio Code	Source-code editing and project development
Web Browser	Running and testing the public website and administrative interface
Node.js	Supporting the JavaScript/TypeScript development environment and server-side functionality
Git	Source-code version control
GitHub	Repository management and team collaboration
Supabase	Database, authentication, storage, and related backend services
PostgreSQL	Relational database management
Vite	Development and build tooling
Playwright	Automated and browser-based testing
TypeScript	Application programming language
React	Frontend user-interface development
TanStack Start	Full-stack application framework
TanStack Router	Application routing
Tailwind CSS	User-interface styling

2. 2.4 Database Resources

The project used PostgreSQL as its relational database and Supabase as the managed platform supporting database-related services. The database was used to store and manage structured information required by the university website, including academic information, website content, leadership information, administrative offices, news, announcements, events, research, services, vacancies, media, users, roles, and system activity records.

Database resources were important for implementing dynamic website functionality because content could be stored and retrieved from the database rather than being maintained only as static page content.

2. 2.5 Authentication and Security Resources

Authentication and security resources were required to protect the administrative part of the system. The project used Supabase Authentication for administrator authentication and PostgreSQL Row-Level Security (RLS) for database-level access control. Sessions, authentication, authorization, and security policies were tested during development.

These resources supported the protection of administrative functionality and helped control access to database operations.

2. 2.6 Development and Testing Resources

Development resources were used throughout the implementation process. The project followed a feature-by-feature approach in which a feature was identified, analyzed, implemented, connected to the database when required, tested, corrected, and reviewed.

Testing resources were used to verify:

Public website pages and navigation.

Administrative functions.

Database operations.

Authentication and authorization.

Media management.

Responsive layouts.

Error handling.

API and database interactions.

TypeScript and ESLint checks.

Application build and integration.

The project used browser-based testing, database verification, API/database checks, and other testing tools as part of the development and verification process.

2. 2.7 Communication and Collaboration Resources

Communication and collaboration were important resources because the project involved coordination between team members. Telegram and in-person communication were used for

discussions and coordination, while Git and GitHub supported source-code management and collaborative development.

These resources helped the team discuss requirements, identify problems, share progress, review changes, and maintain different versions of the project.

2. 3 Risk Management

Risk management was an important part of the development of the Oda Bultum University (OBU) Official Website and Administrative Content Management System. The project involved several technical and organizational activities, including frontend and backend development, database integration, authentication, media management, testing, security verification, and team collaboration. Therefore, potential risks were identified and addressed during the development process.

The project faced technical challenges related to PostgreSQL and Supabase integration, dynamic content management, authentication and protected routes, Row-Level Security (RLS), image uploads, routing, frontend/backend/database inconsistencies, testing, and server/database operations.

2. 3.1 Risk Identification

The major risks considered during the project are presented in Table 2.5.

Table 2.5: Identified Project Risks

No.	Risk	Possible Effect	RiskManagement Approach
1	Database integration problems	Website features may fail to retrieve or save information correctly	Verify database connections and operations during development and testing
2	Frontend and backend inconsistencies	Features may not work correctly when different system components are integrated	Test frontend-backend integration and verify API/database operations
3	Authentication problems	Unauthorized users may access protected administrative functionality	Test authentication, sessions, protected routes, and authorization
4	Incorrect access-control configuration	Users may access data or operations beyond their permissions	Review authorization rules and PostgreSQL RLS policies
5	Media upload problems	Images or other media may fail to upload, display, or be managed correctly	Test media upload and management functionality
6	Routing problems	Users may be unable to access	Test navigation, routes,

No.	Risk	Possible Effect	RiskManagement Approach
		required pages	and page accessibility
7	Dynamic content errors	Website information may not display or update correctly	Verify database records and test content-management operations
8	Software or build errors	The application may fail during development or deployment	Perform development checks, testing, debugging, and build verification
9	Security vulnerabilities	Sensitive administrative or database functionality may be exposed	Perform authentication, authorization, RLS, and security verification
10	Team communication problems	Delays, duplicated work, or misunderstandings may occur	Use regular communication and collaboration through Telegram, in-person discussions, Git, and GitHub
11	Testing and debugging difficulties	Undetected errors may remain in the final system	Perform repeated testing and debugging throughout development
12	Database/server operation errors	Website functionality depending on server or database operations may fail	Test server-side and database operations and verify system behavior

2. 3.2 Technical Risk Management

Database integration was one of the important technical areas requiring risk management. The application used PostgreSQL and Supabase, and database operations were verified during development. Database-related risks were managed by checking data operations and testing the interaction between the application and the database.

Frontend-backend integration was also monitored because inconsistencies between different parts of the application could result in incorrect functionality. Testing and verification were therefore performed after implementing features to identify integration problems and correct them before continuing.

2. 3.3 Authentication and Security Risk Management

Because the system contained a protected administrative content management system, authentication and authorization were important areas of risk management. The project used Supabase Authentication and PostgreSQL Row-Level Security to support access control and database security.

Authentication, authorization, sessions, protected functionality, and database security were tested during development. These tests helped identify access-control issues and verify that security mechanisms were operating as intended.

The security review also identified an area requiring further attention: some administrative server functions were using a direct PostgreSQL connection without validating the caller's Supabase authentication token or session. This was documented as a partially passed security area and identified as requiring correction.

2. 3.4 Testing and Debugging as Risk Control

Testing and debugging were used as major risk-control activities throughout the project. Rather than waiting until the end of development, features were tested after implementation and integration.

The testing process included checking:

Public website pages

Administrative functions.

Navigation and routing.

Database operations.

Authentication and authorization.

Media management.

Responsive behavior.

Error handling.

API and database interactions.

TypeScript and ESLint checks.

Application build and integration.

These activities helped identify errors and inconsistencies and allowed them to be corrected during development.

2. 3.5 Risk Monitoring and Response

Risk management was not treated as a one-time activity. Risks were monitored during the different stages of development. When a problem was identified, the development process

involved analyzing the issue, implementing a correction, testing the correction, and reviewing the result.

The general development process followed the sequence of identifying a feature or issue, analyzing it, implementing the required solution, connecting the relevant database functionality, testing, correcting errors, reviewing the result, and integrating the completed work.

This approach reduced the possibility of carrying unresolved technical problems into later development stages.

2. 3.6 Risk Management Summary

The major risk-management activities used in the OBU website project can be summarized as follows:

1. Early identification of technical problems during system analysis and development.

2. Continuous testing and debugging rather than testing only at the end.

3. Database verification to confirm that application data operations functioned correctly.

4. Authentication and authorization testing for protected administrative functions.

5. Security verification of PostgreSQL RLS and other access-control mechanisms.

5. Frontend-backend integration testing to identify inconsistencies between system components.

6. Media and routing testing to verify important website functionality.

7. Git and GitHub version control to support collaboration and manage source-code changes.

8. Regular communication to identify and resolve development issues.

9. Final review and verification before considering the implemented functionality complete.

2. 4 Development Methodology

The Oda Bultum University (OBU) Official Website and Administrative Content Management System was developed using an iterative and incremental development approach. The development process was organized around individual features and functional areas. Each feature was identified, analyzed, implemented, connected to the database where necessary, tested, corrected, reviewed, and then integrated into the overall system.

This approach was suitable for the project because the OBU website contained several different functional areas, including academic information, departments and programs, leadership and administrative offices, registrar services, research, community engagement, vacancies, news, announcements, events, galleries, contact information, and administrative content management.

2. 4.1 Development Approach

The development process followed a continuous cycle for each major feature:

Identify → Analyze → Design/Implement → Integrate → Test → Correct → Review → Integrate into System

The main stages are described below.

1. Feature Identification

The first step was to identify the feature or functionality that needed to be developed or improved. Requirements were considered according to the purpose of the website and the needs of its public users and administrators.

2. Analysis

After identifying a feature, the existing project structure and related functionality were analyzed. This included examining frontend components, backend functionality, database requirements, routing, authentication, and other related parts of the system.

3. Design and Implementation

The required frontend and backend functionality was then implemented. Frontend development involved pages, reusable components, navigation, layouts, and responsive interfaces. Backend development supported server-side operations and communication with the database.

The project used technologies including TypeScript, React, TanStack Start, TanStack Router, Tailwind CSS, and Node.js.

4. Database Integration

When a feature required persistent data, it was connected to the PostgreSQL database and related Supabase services. The database supported information such as academic content, website content, leadership, administrative offices, news, announcements, events, research, services, vacancies, media, users, roles, and system activity records.

5. Testing

After implementation and integration, the feature was tested to determine whether it operated as expected. Testing covered public website functionality, administrative functions, database

operations, authentication, media management, navigation, responsive behavior, and other system operations.

6. Correction and Debugging

If errors or unexpected behavior were identified during testing, the relevant code or configuration was corrected. The feature was then tested again to verify that the problem had been resolved.

7. Review and Integration

After successful testing and correction, the completed feature was reviewed and integrated with the rest of the application. This process was repeated for subsequent features.

2. 4.2 Application of the Methodology

The iterative approach was applied to different areas of the OBU website and CMS. These included:

Public website pages and navigation.

Academic information and university organizational content.

Leadership and administrative information.

Registrar and student-related services.

Research and community engagement information.

News and announcements.

Events and galleries.

Vacancies and career information.

Media management.

Contact functionality.

Administrative content management.

User and role management.

Authentication and authorization.

This allowed individual functional areas to be developed and tested before being combined into the complete website and CMS.

2. 4.3 Database and Security Integration

The methodology also incorporated database and security verification into the development cycle. PostgreSQL was used as the relational database, while Supabase provided managed database and related backend services. Supabase Authentication was used for administrator authentication, and PostgreSQL Row-Level Security (RLS) was used as part of the database access-control mechanism.

Security was therefore considered during development and testing rather than being treated only as a final-stage activity. Authentication, authorization, database access, and RLS-related behavior were included in verification activities.

2. 4.4 Why the Iterative Approach Was Appropriate

The iterative approach was appropriate for the OBU website project because the system contained many functional areas and required continuous interaction between frontend development, backend functionality, database operations, authentication, media management, and testing.

The approach provided the following benefits:

Incremental development: Features could be developed progressively rather than requiring the complete system to be implemented at once.

Early testing: Features could be tested shortly after implementation.

Continuous debugging: Problems could be identified and corrected during development.

Database verification: Database-dependent features could be checked as they were implemented.

Continuous integration: Completed features could be reviewed and integrated into the wider application.

Flexibility: Development could continue feature by feature as different requirements and technical issues were identified.

Improved verification: Repeated testing allowed implemented functionality to be checked before moving to subsequent work.

The project documentation specifically describes development as a feature-by-feature process involving identification, analysis, implementation, database connection, testing, correction, review, and integration.

2. 5 Quality Assurance

Quality Assurance (QA) was an important part of the development of the Oda Bultum University (OBU) Official Website and Administrative Content Management System. QA activities were performed throughout development to verify that implemented features operated correctly, were properly integrated, and met the intended functional and technical requirements.

The project followed a continuous testing and verification approach. Features were implemented and then tested, errors were identified and corrected, and the results were reviewed before further integration. This approach helped reduce errors and improve the reliability of the completed system.

2. 5.1 Quality Assurance Objectives

The main objectives of quality assurance were to:

Verify that website features functioned according to their intended requirements.

Ensure that public pages and administrative functions operated correctly.

Verify frontend, backend, and database integration.

Identify and correct errors during development.

Verify authentication and authorization functionality.

Check database security and access-control mechanisms.

Verify media upload and management functionality.

Test navigation and routing.

Check responsive behavior across different screen sizes.

Verify error handling and system behavior.

Maintain source-code quality through development checks and version control.

2. 5.2 Testing Activities

Different testing and verification activities were performed during the project. The documented testing activities included public and administrative functionality testing, database operation verification, authentication testing, media testing, responsive testing, error checking, API/database verification, and TypeScript, ESLint, and build checks.

Table 2.6: Quality Assurance and Testing Activities

Testing Area	Quality Assurance Activity	Purpose
Public Website Testing	Test public pages, navigation, and displayed content	Verify that visitors can access and use website features
Administrative Testing	Test CMS functions and administrative operations	Verify that administrative features operate correctly
Database Testing	Verify database operations and stored information	Ensure correct data retrieval and modification
Authentication Testing	Test administrator authentication and sessions	Verify protected access
Authorization Testing	Test access-control behavior	Verify that protected functionality is appropriately controlled
Media Testing	Test image/media upload and management	Verify media functionality
Responsive Testing	Test interfaces on different screen sizes	Verify responsive behavior
Routing Testing	Test navigation and application routes	Ensure required pages can be accessed
API/Database Verification	Check communication between application and database	Identify integration problems
Code Quality Checks	TypeScript, ESLint, and build verification	Identify code and build-related problems
Error Testing	Observe and correct unexpected system behavior	Improve reliability and stability
Security Verification	Review authentication, authorization, and RLS behavior	Identify potential security problems

2. 5.3 Frontend Quality Assurance

Frontend quality was verified through testing of pages, components, navigation, layouts, and responsive interfaces. The project used React, TypeScript, TanStack Start, TanStack Router, and Tailwind CSS as major frontend and application technologies.

Frontend testing focused on whether:

Pages loaded correctly.

Navigation links directed users to the intended pages.

Content was displayed correctl

Components behaved as expecte

Layouts adapted to different screen sizes.

User interactions worked correctl

Errors were identified and corrected.

The intern's documented contribution included frontend pages and components, navigation, responsive layouts, frontend-backend integration, and user-facing testing.

2. 5.4 Backend and Database Quality Assurance

Backend and database functionality was tested to verify that application operations interacted correctly with the database. PostgreSQL was used as the relational database, with Supabase providing managed database and related backend services.

Quality assurance activities included verifying:

Database connectivity.

Data retrieval and storage.

Database-dependent website features.

API and database interactions.

Administrative content operations.

Consistency between application functionality and stored data.

These checks helped identify problems that could occur when frontend functionality, backend operations, and database structures were integrated.

2. 5.5 Authentication and Security Quality Assurance

Authentication and security were important quality requirements because the system included protected administrative functionality. Supabase Authentication was used for administrator authentication, while PostgreSQL Row-Level Security (RLS) was used as part of the database security and access-control mechanism.

Testing and verification were performed on authentication, authorization, sessions, protected functionality, and database security.

The security review also identified a limitation requiring further correction: some administrative server functions directly used a PostgreSQL connection without validating the caller's Supabase authentication token or session. This area was documented as partially passed and requiring correction.

Recording this issue was itself an important QA activity because it prevented the security review from treating an unresolved issue as fully completed.

2. 5.6 Code Quality and Development Checks

Code quality was supported through development checks involving TypeScript, ESLint, and application build verification. These checks helped identify programming errors, code-quality problems, and issues that could prevent the application from building or operating correctly.

Git and GitHub were also used for source-code version control and collaboration. This supported the management of code changes and coordination among team members during development.

2. 5.7 Documentation and Review

Documentation and review were included as part of the overall development process. After implementation and testing, features were reviewed before being integrated into the wider system. The development process therefore combined implementation, testing, correction, review, and integration rather than treating quality assurance as a single final activity.

Project documentation also supported understanding of the system, its technologies, functionality, testing activities, and identified issues.

2. 5.8 Quality Assurance Process

The overall QA process can be summarized as:

Requirement → Implementation → Integration → Testing → Error Identification → Correction → Retesting → Review → Final Integration

This process was repeated for different features and functional areas of the OBU website and CMS.

The approach allowed problems to be identified earlier and addressed during development. It also supported continuous verification of the public website, administrative CMS, database operations, authentication, media functionality, and security mechanisms.

2. 5.9 Quality Assurance Summary

Quality assurance was integrated throughout the development of the OBU website rather than being performed only after implementation was completed. Testing covered the public website, administrative CMS, database operations, authentication, authorization, media management, navigation, responsive behavior, API/database interactions, and code-quality checks.

The combination of continuous testing, debugging, database verification, security testing, code-quality checks, version control, and review helped improve the reliability and functionality of the system. At the same time, the security review documented an unresolved area involving

authentication-token/session validation in some administrative server functions, providing a clear item for further improvement.

2.6 Required Project Documents
The main documents relevant to the project include the project schedule and milestones, system requirements, system models and diagrams, database design documentation, interface and navigation descriptions, implementation notes, test records, and user guidance for public and administrative functions. These documents support project planning, development, verification, maintenance, and future improvement. Any document not completed during the internship should be identified as pending rather than reported as finished.

Chapter 3: System Requirements

3. 1 Existing System Analysis

The existing system analysis was conducted to identify the limitations and requirements of the university website before developing the proposed Oda Bultum University (OBU) Official Website and Administrative Content Management System (CMS). The analysis focused on website content, information organization, content management, security, media management, and database functionality.

3. 1.1 Overview of the Existing System

The existing website provided university information to students, staff, prospective students, researchers, job seekers, and the general public. However, managing and updating different types of information was difficult. The project therefore required a dynamic website with a centralized administrative system for managing institutional content.

3. 1.2 Problems Identified

The main problems identified were:

Difficulty maintaining and updating website content.

Incomplete or poorly organized information.

Limited centralized content management functionality.

Difficulty managing dynamic information such as news, events, and announcements.

Need for better organization and navigation of university information.

Need for secure administrative authentication and authorization.

Requirements for image and media management.

Challenges in integrating the frontend, backend, database, and authentication services.

3. 1.3 Existing System Limitations

Table 3.1: Existing System Limitations

No.	Limitation	Effect
1	Difficult content maintenance	Information updates require more technical effort
2	Poorly organized information	Users may have difficulty finding information
3	Limited CMS functionality	Administrators have less control over content
4	Dynamic content challenges	News and events are difficult to manage efficiently
5	Media management requirements	Images require proper upload and management
6	Security requirements	Administrative functions need access control
7	Integration challenges	Frontend, backend, and database inconsistencies may affect functionality

3. 1.4 Need for Improvement

The analysis showed the need for a dynamic and centralized university website that supports effective content management, organized information, secure administrative access, media management, and database-driven functionality. These findings were used to define the requirements of the proposed system.

3. 2 Proposed System

The proposed system is the Oda Bultum University (OBU) Official Website and Administrative Content Management System (CMS). It combines a public university website with a protected administrative system for managing website content.

The public website provides information for students, prospective students, staff, researchers, job seekers, and the general public. The CMS allows authorized administrators to create, update, and manage university information without directly modifying the source code or database.

3. 2.1 Main Features

The proposed system provides:

Academic and administrative information.

News, announcements, and events.

Research and community engagement information.

University services and vacancies.

Media and gallery management.

Contact and inquiry services.

User and role management.

Authentication and authorization.

Database-driven content management.

3. 2.2 System Architecture

The system follows a full-stack architecture consisting of:

Frontend: React, TypeScript, TanStack Start, TanStack Router, and Tailwind CSS.

Application Layer: Node.js and the full-stack application framework.

Database: PostgreSQL with Supabase services.

Security: Supabase Authentication and PostgreSQL Row-Level Security (RLS).

The basic system flow is:

User → Web Interface → Application Server → PostgreSQL/Supabase Database

3. 2.3 Administrative CMS

The CMS provides authorized administrators with facilities to manage academic information, departments, programs, leadership, news, announcements, events, media, research, services, vacancies, users, and roles.

3. 2.4 Security

Administrative functions are protected using authentication and authorization. Supabase Authentication manages administrator authentication, while sessions and PostgreSQL RLS help control access to protected functions and database information.

3. 2.5 Expected Benefits

The proposed system provides:

Centralized content management.

Organized university information.

Dynamic database-driven content.

Secure administrative access.

Media management.

Improved navigation and accessibility.

Easier future maintenance and expansion.

3. 3 Functional Requirements

Functional requirements describe the main functions that the OBU Official Website and Administrative Content Management System must provide. The system consists of a public website and a protected administrative CMS.

The main functional requirements are:

1. Public Website

    View university information.

    View academic, leadership, registrar, research, and service information.

    View news, announcements, events, vacancies, and galleries.

    Access contact information.

2. Authentication and Authorization

    Administrator login and logout.

    Manage authenticated sessions.

    Restrict CMS functions to authorized users.

    Manage user roles and permissions.

3. Content Management

    Create, update, and manage academic information.

    Manage departments and programs.

Manage news, announcements, events, research, services, and vacancies.

Manage contact-related information.

4. Media Management

Upload, manage, and display website images and other media.

5. Database Management

Store and retrieve university information.

Update database records through the CMS.

Maintain relationships between related data.

6. Navigation

Provide navigation between public website sections.

Provide navigation between administrative functions.

7. Security and Testing

Apply authentication and authorization.

Use database access control and Row-Level Security (RLS).

Test system functions, database operations, authentication, routing, and integration.

Table 3.2: Summary of Functional Requirements

Category	**Main Functions**
Public Website	University and academic information
Content	News, announcements, events, vacancies
CMS	Create, update, and manage content
Authentication	Login, sessions, logout
User Management	Users, roles, permissions
Media	Upload and manage media
Database	Store, retrieve, and update data
Navigation	Public and administrative routing

Category	Main Functions
Security	Authentication, authorization, RLS
Testing	Functional and integration testing

Overall, the system must allow public users to access organized university information while authorized administrators securely manage dynamic content through the CMS.

- Visitors can view approved university information
- Administrators can sign in securely
- Authorized staff can create and update selected content
- News, events, media, and vacancies can be managed where implemented
- Roles restrict administrative actions

3. 4 Non-Functional Requirements

Non-functional requirements describe the quality and operational characteristics of the proposed OBU website and CMS. The main requirements are:

1. Usability: The system should be easy to understand, navigate, and use for both public and administrative users.

2. Reliability: The system should operate correctly and consistently, including database operations, authentication, and content management.

3. Performance: Web pages, database operations, and administrative functions should provide reasonable response times during normal use.

4. Security: The system should protect administrative functions through authentication, authorization, sessions, and database access control.

5. Accessibility: Information should be presented clearly and remain usable for different categories of users and screen sizes.

6. Maintainability: Website content and system components should be easy to update and maintain.

7. Data Integrity: Stored information should remain accurate, consistent, and properly related within the database.

8. Scalability: The system should support additional content and future improvements.

9. Legal and Institutional Compliance: The system should follow applicable university policies and relevant requirements for managing and publishing information.

Table 3.3: Non-Functional Requirements

Requirement	Description
Usability	Easy to use and navigate
Reliability	Consistent and correct operation
Performance	Reasonable response time
Security	Authentication and access control
Accessibility	Clear and accessible information
Maintainability	Easy to update and maintain
Data Integrity	Accurate and consistent data
Scalability	Supports future expansion
Compliance	Follows institutional requirements

- Responsive interface for mobile, tablet, and desktop
- Usable and accessible interface
- Secure authentication and authorization
- Reliable database and media storage
- Maintainable code and documentation

Chapter 4 System Analysis and Design

4. 1 System Analysis and Design Overview

System analysis and design describes how the requirements identified in Chapter Three were transformed into the structure of the Oda Bultum University (OBU) website and Administrative Content Management System (CMS).

The proposed system consists of a public website and a protected administrative CMS. The public website provides university information to students, staff, researchers, prospective students, and the general public. The CMS allows authorized administrators to manage website content.

The main components of the system are:

Public Website: Presents academic, administrative, news, events, research, services, and other university information.

Administrative CMS: Allows authorized users to create, update, and manage website content.

Application Layer: Handles system operations and communication between the interface and database.

Database: PostgreSQL stores university and application information.

Authentication and Security: Supabase Authentication and PostgreSQL RLS help protect administrative functions and data.

Media Management: Supports uploading and displaying website images and other media

The basic system interaction is:

User → Web Interface → Application → PostgreSQL/Supabase Database → Website

For administrators:

Administrator → Authentication → CMS → Application → Database

The design also considers system models, user interface design, software architecture, and security/access control. These components provide the foundation for implementing and testing the proposed system.

4. 2 System Models

System models show the main users, functions, processes, and interactions of the Oda Bultum University (OBU) website and Administrative CMS. The models help explain how the public website, CMS, application, authentication, and database work together.

4. 2.1 System Actors

The main actors are:

Public User: Views university information, academic programs, news, events, vacancies, services, galleries, and submits contact messages.

Administrator: Logs into the CMS and manages website content, users, media, news, events, and other information.

Authentication Service: Handles administrator authentication and access.

Database: Stores website content, users, roles, media, and other system data.

4. 2.2 Use Case Model

The use case model describes the main activities performed by public users and administrators.

Public users can view university information, academic programs, leadership, research, services, vacancies, news, announcements, events, galleries, and submit contact inquiries.

Administrators can log in and manage academic information, departments, programs, leadership, news, announcements, events, vacancies, research, services, media, and other website content.

A simple relationship is:

Administrator → CMS → Database → Updated Website Content

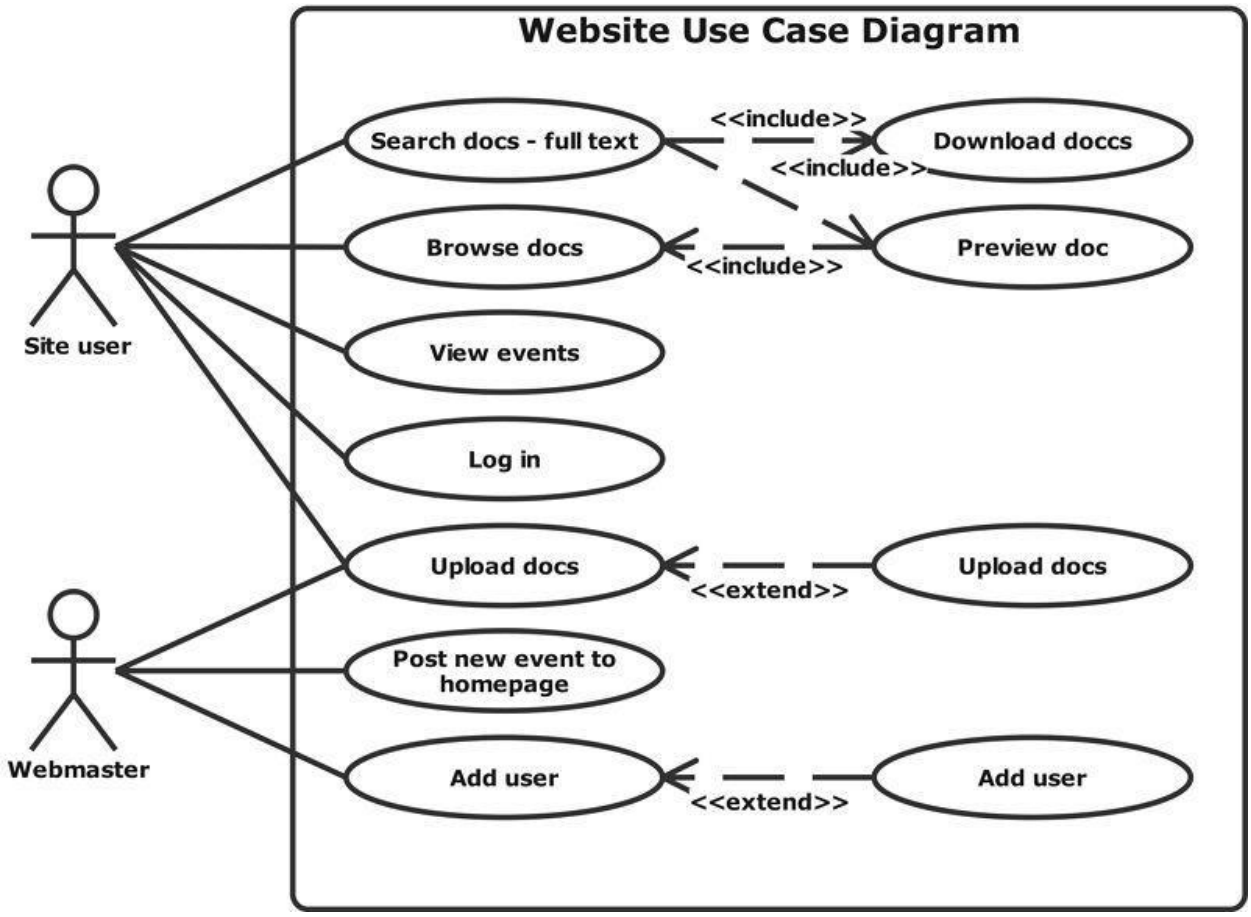


Figure 4.1: Use Case Diagram of OBU Website and Administrative CMS

4. 2.3 Activity Model

The activity model shows the flow of common system operations.

Public user:

Open Website → Select Section → Request Information → Retrieve Data → Display Information

Administrator:

Login → Authentication → Access CMS → Manage Content → Save Changes → Display Updated Content

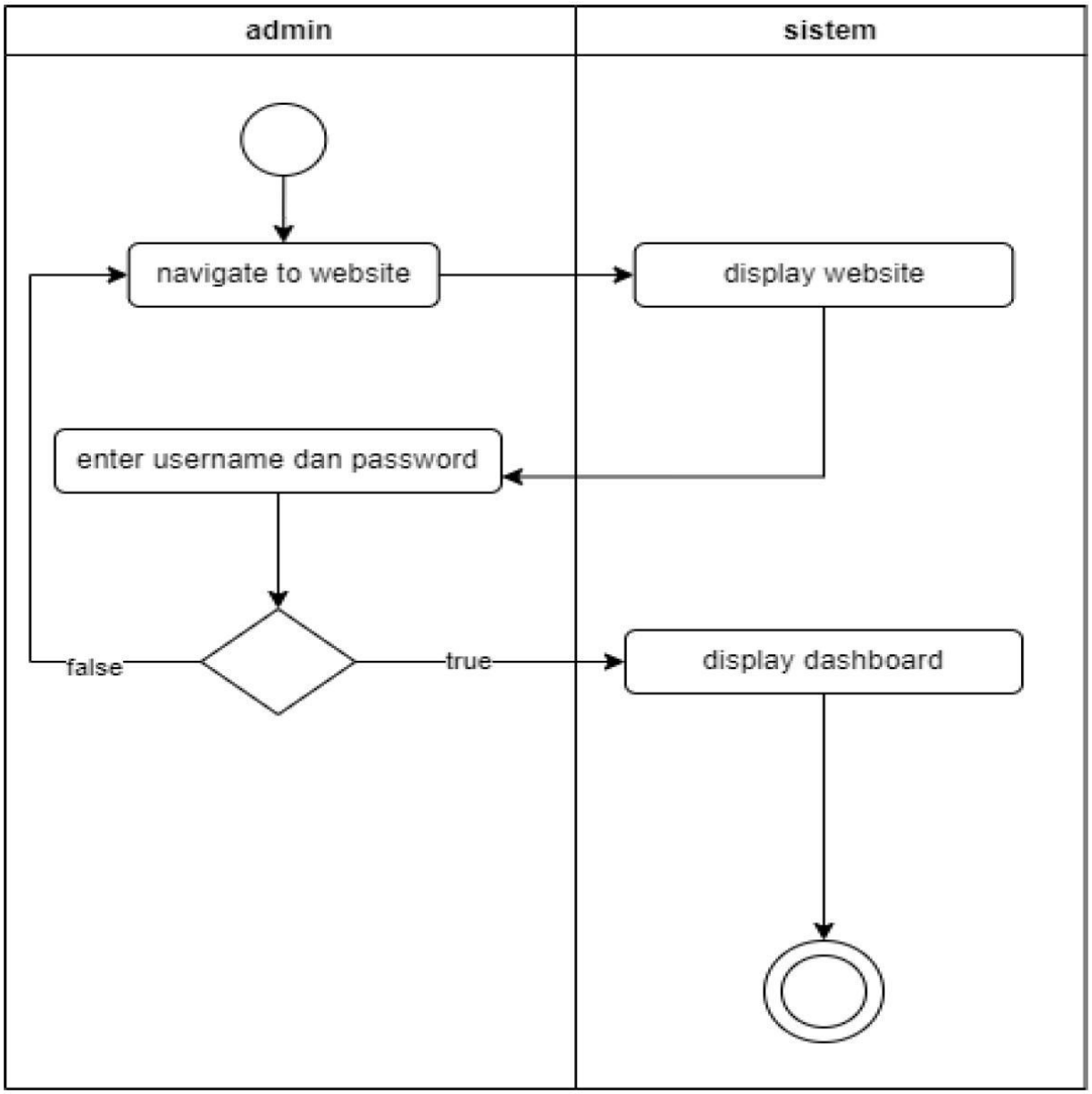


Figure 4.2: Activity Diagram of Administrator Login and Content Management

4. 2.4 Sequence Model

The sequence model shows how the main components communicate.

For public users:

User → Website → Application → Database → Website → User

For administrators:

Administrator → CMS → Authentication → Application → Database → CMS

These sequences show how requests and data move between the different parts of the system.

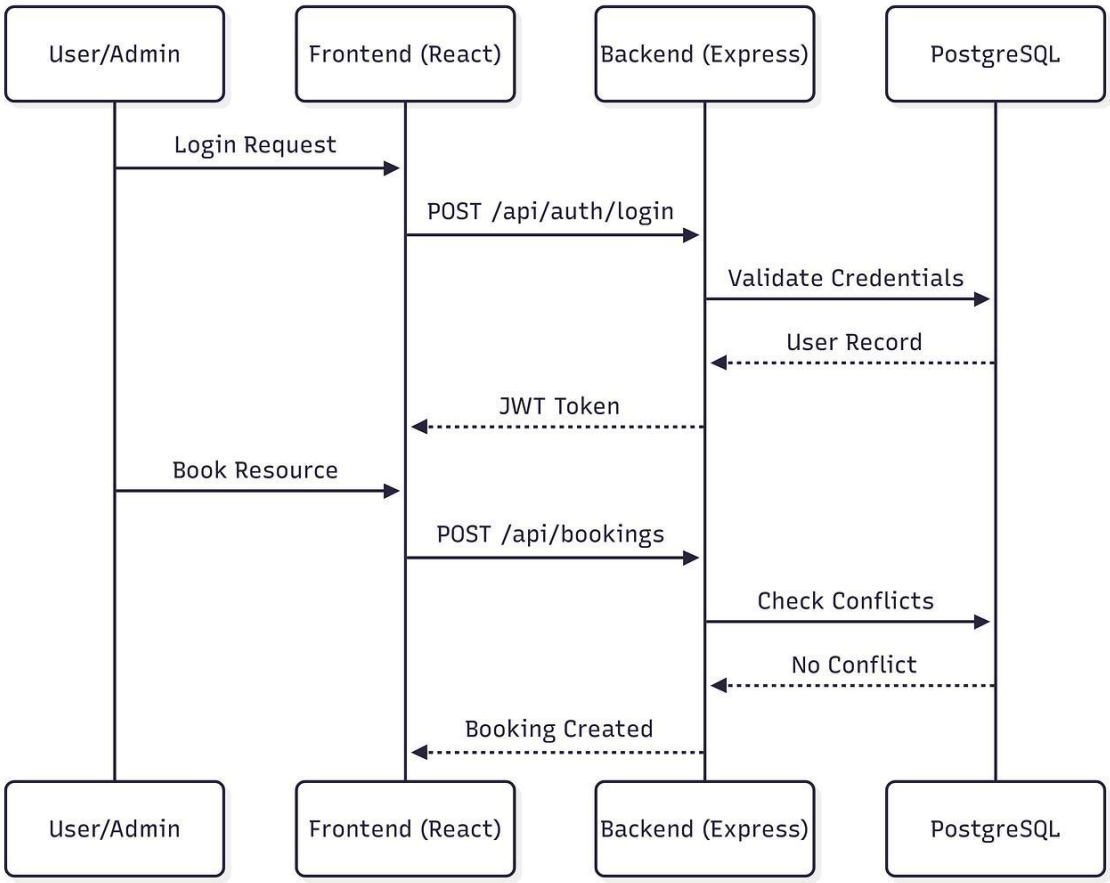


Figure 4.3: Sequence Diagram of the OBU Website and Administrative CMS

4. 2.5 Class/Object Model

The class/object model represents the main data entities used by the system. These include:

Users and roles

Academic units, departments, and programs

News and announcements

Events and vacancies

Research and services

Media and website content

Some basic relationships are:

User → Role → Permission

Academic Unit → Department → Program

Administrator → CMS Operation →
Databas

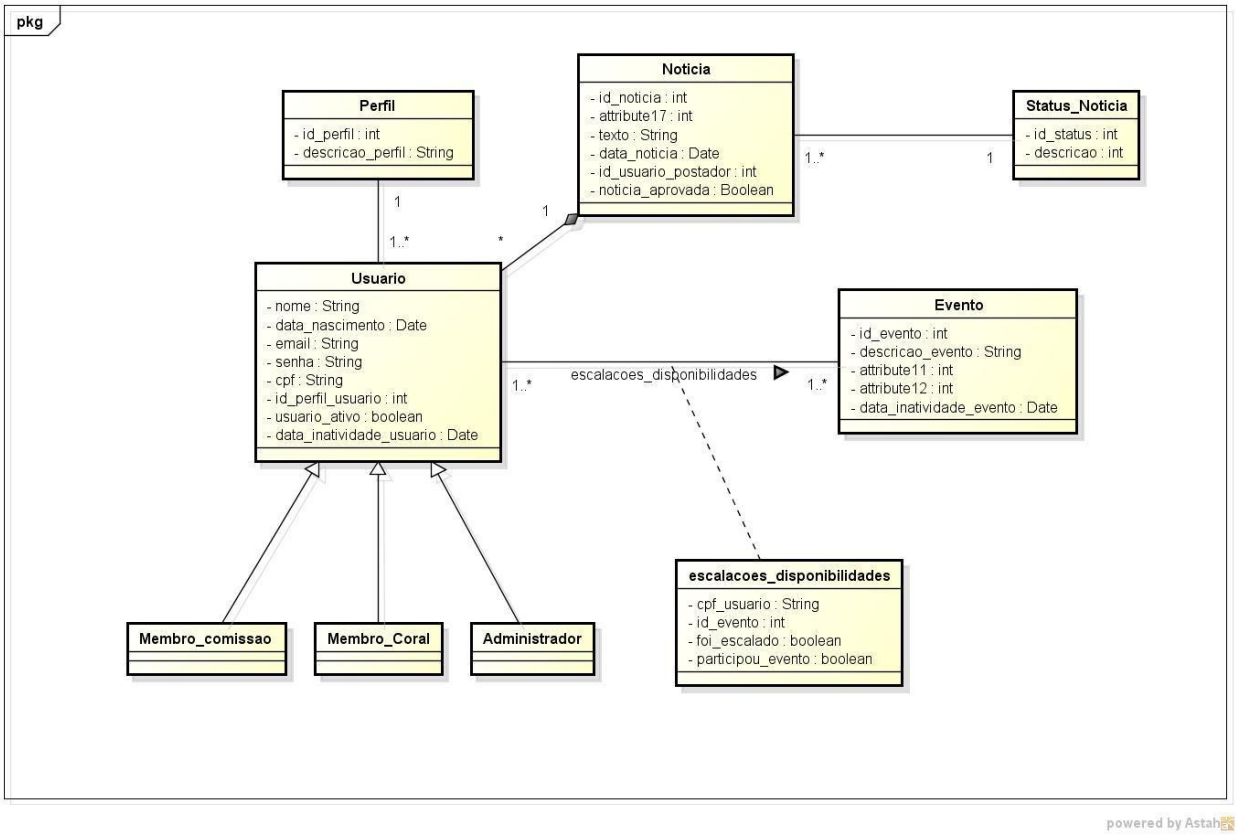


Figure 4.4: Class/Object Diagram of the OBU Website and Administrative CMS

4. 2.6 Model Integration

The system models describe different parts of the same system:

Use Case Model: Shows what users can do.

Activity Model: Shows the flow of activities.

Sequence Model: Shows the order of system interactions.

Class/Object Model: Shows important entities and their relationships.

Together, these models provide a clear view of the system before implementation.

4. 3 User Interface Design and Navigational Flow

The user interface (UI) was designed to make the Oda Bultum University website easy to use and to provide administrators with a simple way to manage website content. The system has two main interfaces: the public website and the administrative CMS.

4. 3.1 Public Website Interface

The public website provides access to university information such as:

University and academic information

Departments and programs

Leadership and administrative offices

Research and services

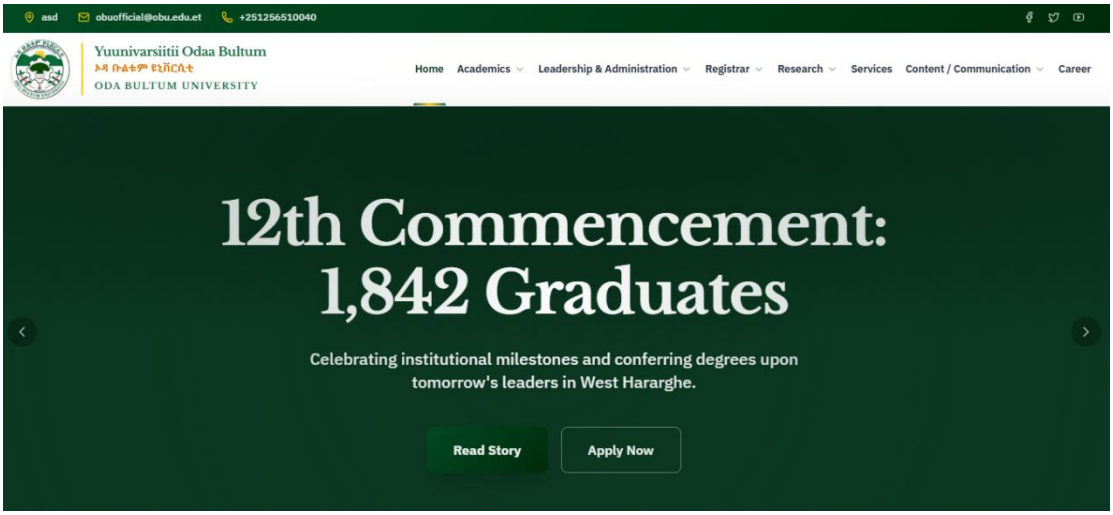
News, announcements, and events

Vacancies and galleries

Contact information

The homepage acts as the main entry point and provides navigation to the different sections of the website.

OBU University Website Homepage



4. 3.2 Navigation Design

The main public navigation can be summarized as:

Homepage → University Information → Academic Information → Research & Services → News/Events → Contact

Academic information follows:

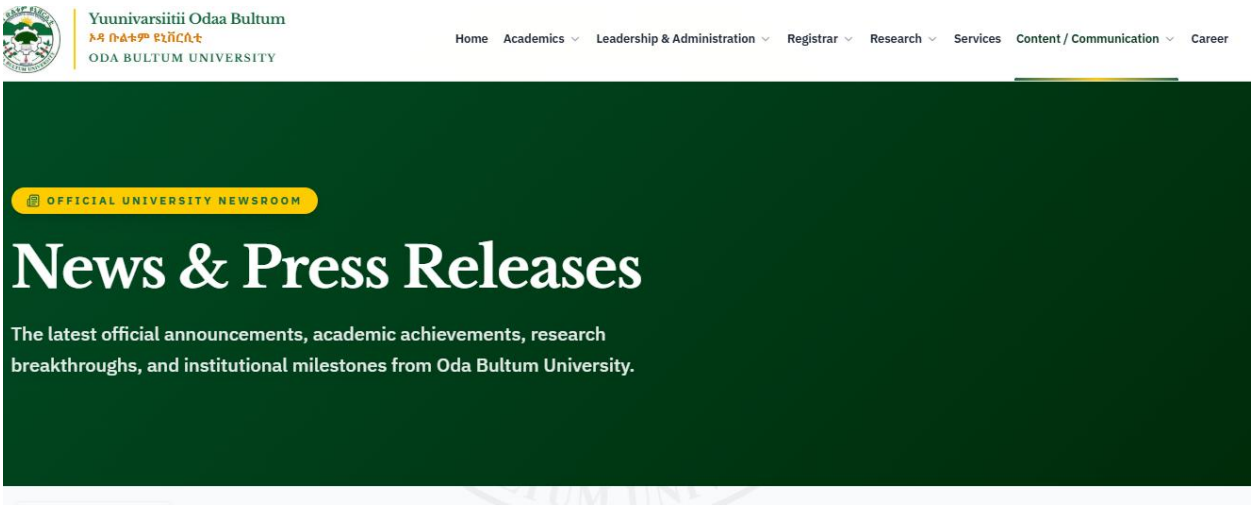
Academic Unit → Department → Program

This structure helps users find information without directly accessing the database or source code.

4. 3.3 News, Events, and Media

News, announcements, events, and galleries are displayed through the public website. Administrators manage this information through the CMS, and the updated data is stored in the database before being displayed to visitors.

CMS → Update Content → Database → Public Website



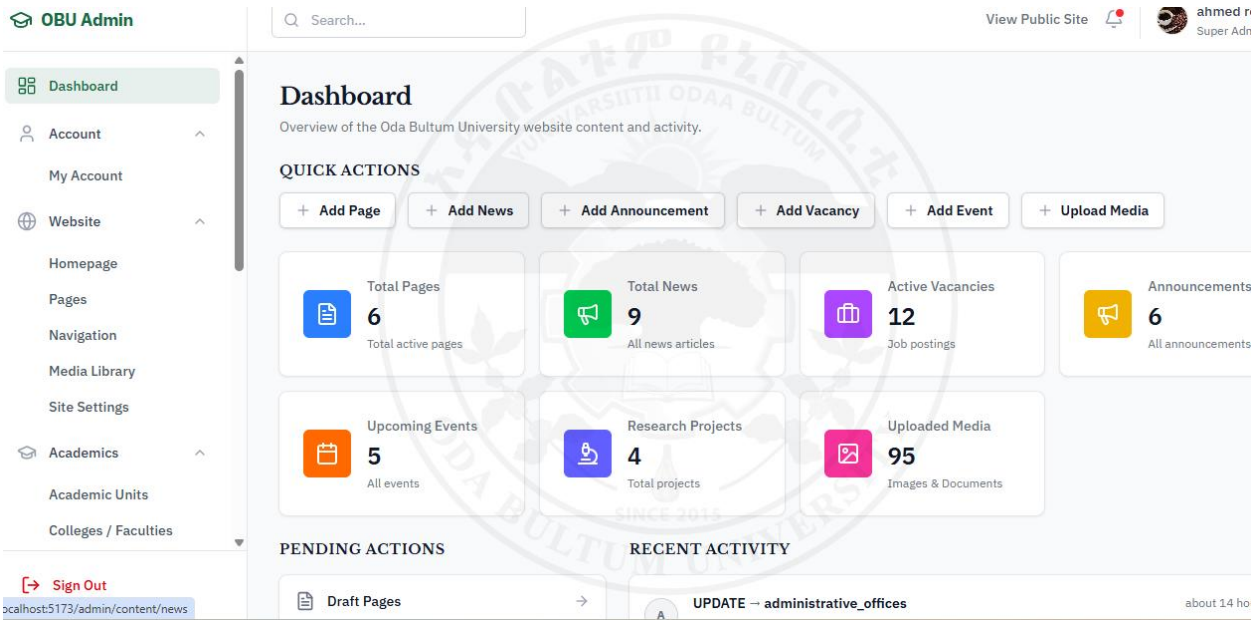
4. 3.4 Administrative CMS Interface

The CMS is a protected area for authorized administrators. It provides functions for managing academic information, departments, programs, leadership, news, announcements, events, research, services, vacancies, media, and users.

The general flow is:

Login → Authentication → CMS Dashboard → Manage Content → Save → Database

Figure 4.5: Administrative CMS Dashboard



4. 3.5 Forms and Media Management

CMS forms allow administrators to create and update website information. Media management is used to upload and manage images used on the website.

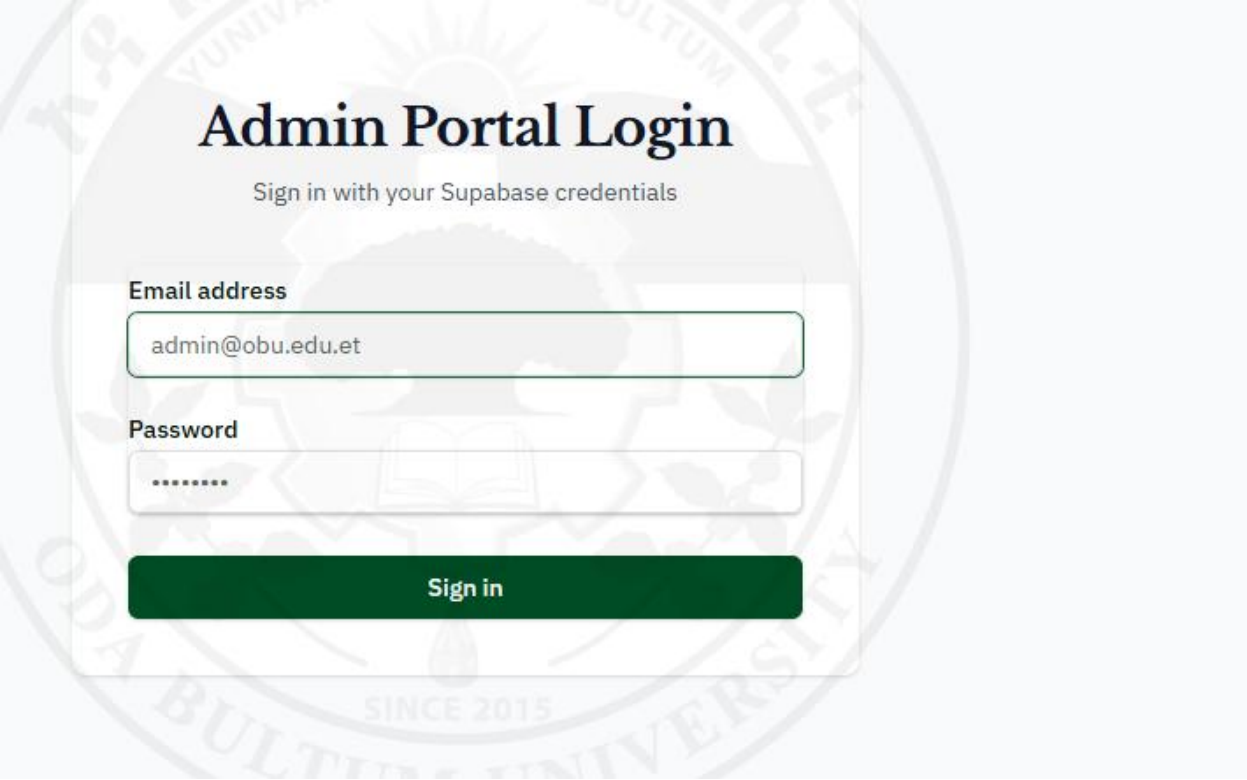
Content:
Open Form → Enter Information → Submit → Database

Media:
Upload Media → Storage → Website Display

4. 3.6 Authentication and Responsive Design

The CMS requires administrator authentication before protected functions can be accessed.

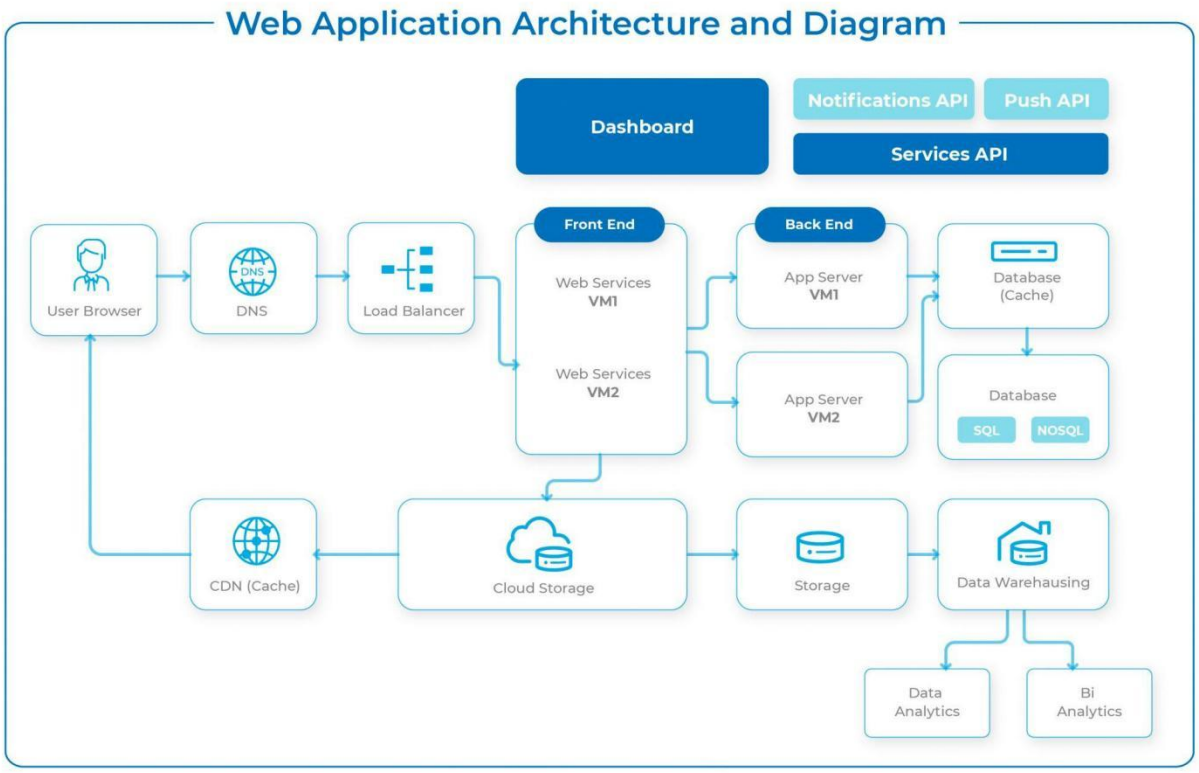
Figure 4.6: Administrative Login Page



The website was also designed with responsive behavior so that pages, navigation, text, and images can be used on different screen sizes.

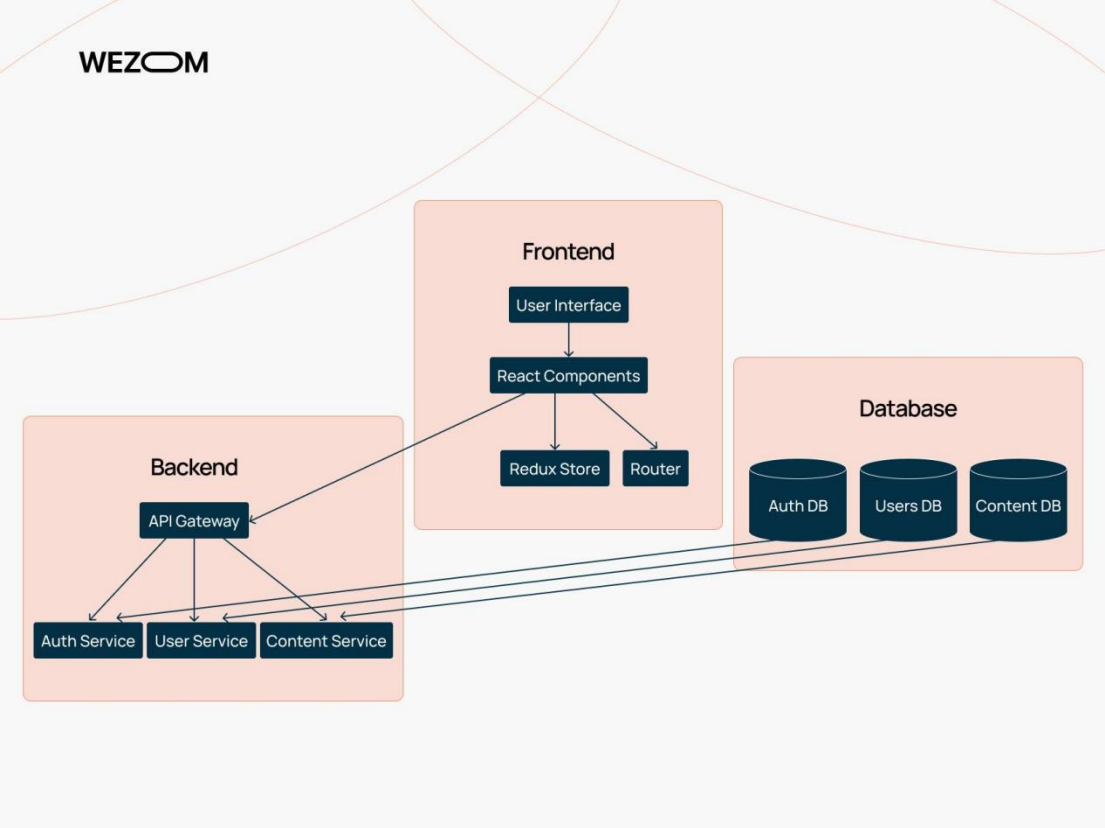
4. 4 Software Architecture and Data Management Design

Software architecture shows how the main parts of the OBU website and Administrative CMS are organized and connected. The system separates the public website from the protected CMS while using common application and database services.



4. 4.1 System Architecture

The main architecture consists of the user interface, application/server layer, authentication, PostgreSQL database, and media services.



The main technologies used in the project include React, TypeScript, TanStack Start, TanStack Router, Tailwind CSS, Node.js, PostgreSQL, and Supabase.

4. 4.2 Data Management

PostgreSQL stores the main university and application information. The database includes academic information, departments, programs, leadership, news, announcements, events, services, vacancies, research, media, users, and roles.

The basic academic relationship is:

Academic Unit → Department → Program

Content can also be organized as:

Content Category → Content → Media

4. 4.3 Data Flow

For public users:

User → Website → Application → Database → Website → User

For administrators:

Administrator → CMS → Authentication → Application → Database → CMS

This allows administrators to update information through the CMS and makes the updated content available on the public website.

4. 4.4 Authentication and Security

Administrative access is protected through authentication and authorization. The project used Supabase Authentication and PostgreSQL Row-Level Security (RLS) as part of its security approach.

4. 4.5 Media Management

The system also supports uploading and managing website images and other media.

Administrator → CMS → Media Upload → Storage → Website Display

4. 5 Security and Access Control Design

Security was an important part of the Oda Bultum University website and CMS. The system separates public pages from protected administrative functions and uses authentication, authorization, sessions, and database security to control access.

4. 5.1 Authentication and Authorization

Supabase Authentication was used to verify administrative users. After successful login, a session is created and the user can access the functions allowed by their role. Public users cannot access the protected CMS.

4. 5.2 Protected Administrative Access

The CMS is separated from the public website. Administrative pages and content-management operations require authentication and authorization before they can be used.

4. 5.3 Database Security

PostgreSQL Row-Level Security (RLS) provides an additional layer of protection for database records. It helps control which users or operations can access or modify data.

4. 5.4 Session Management

Sessions maintain the user's authenticated state after login. Protected operations should only be available when a valid authenticated session is present.

4. 5.5 Security Testing

Security testing included login, protected pages, database operations, authentication, and error handling. These tests helped identify problems between the application, authentication, and database layers.

4. 5.6 Security Limitation

The project review identified that some server-side functions accessed PostgreSQL without checking the user's Supabase authentication token or session. This was recorded as a security issue that requires correction.

4. 5.7 Security Structure

The main security flow can be represented as:

User → Authentication → Session → Authorization → CMS → Database/RLS

Public users follow:

Public User → Public Website → Public Content

Administrators follow:

Administrator → Login → Authentication → CMS → Database

4. 6 Deployment and System Infrastructure Design

The deployment design describes the environment and services used to develop, test, and run the Oda Bultum University website and CMS. The system uses a web interface, application functions, database services, authentication, and storage.

4. 6.1 Development Environment

The project was developed and tested using Visual Studio Code, Node.js, React, TypeScript, TanStack Start/Router, Tailwind CSS, PostgreSQL, Supabase, Git, GitHub, and web browsers. These tools supported coding, database management, version control, and system testing.

4. 6.2 System Deployment Structure

The main system structure can be represented as:

User Browser → Website/CMS → Application Functions → Supabase → PostgreSQL / Authentication / Storage

The public website provides information to visitors, while the CMS provides protected functions for administrators.

4. 6.3 Database and Services

PostgreSQL was used to store the system data, with Supabase providing database, authentication, and storage services. The database contains information such as academic units, programs, news, events, services, users, and other website content.

4. 6.4 Testing and Version Control

The system was tested using web browsers and development tools. Testing covered website pages, CMS functions, authentication, database operations, media, navigation, and responsive behavior. Git and GitHub were used to manage project changes and support collaboration.

4. 6.5 Deployment Workflow

The development process followed these main steps:

Requirements → System Development → Local Testing → Database Verification → Security Testing → Problem Correction → Final Verification

4. 7 Database Design

The database is an important part of the Oda Bultum University website and CMS. It stores the information used by the public website and administrative system. The project uses PostgreSQL, with Supabase providing the database and related services.

The database design describes how the system organizes, stores, and relates information. The Oda Bultum University Official Website and Administrative Content Management System uses PostgreSQL, managed through Supabase, to store website content and related information.

The database supports the management of users, roles, academic units, programs, news, events, media, galleries, website pages, and contact messages. Relationships between tables help maintain data consistency and connect related

informati

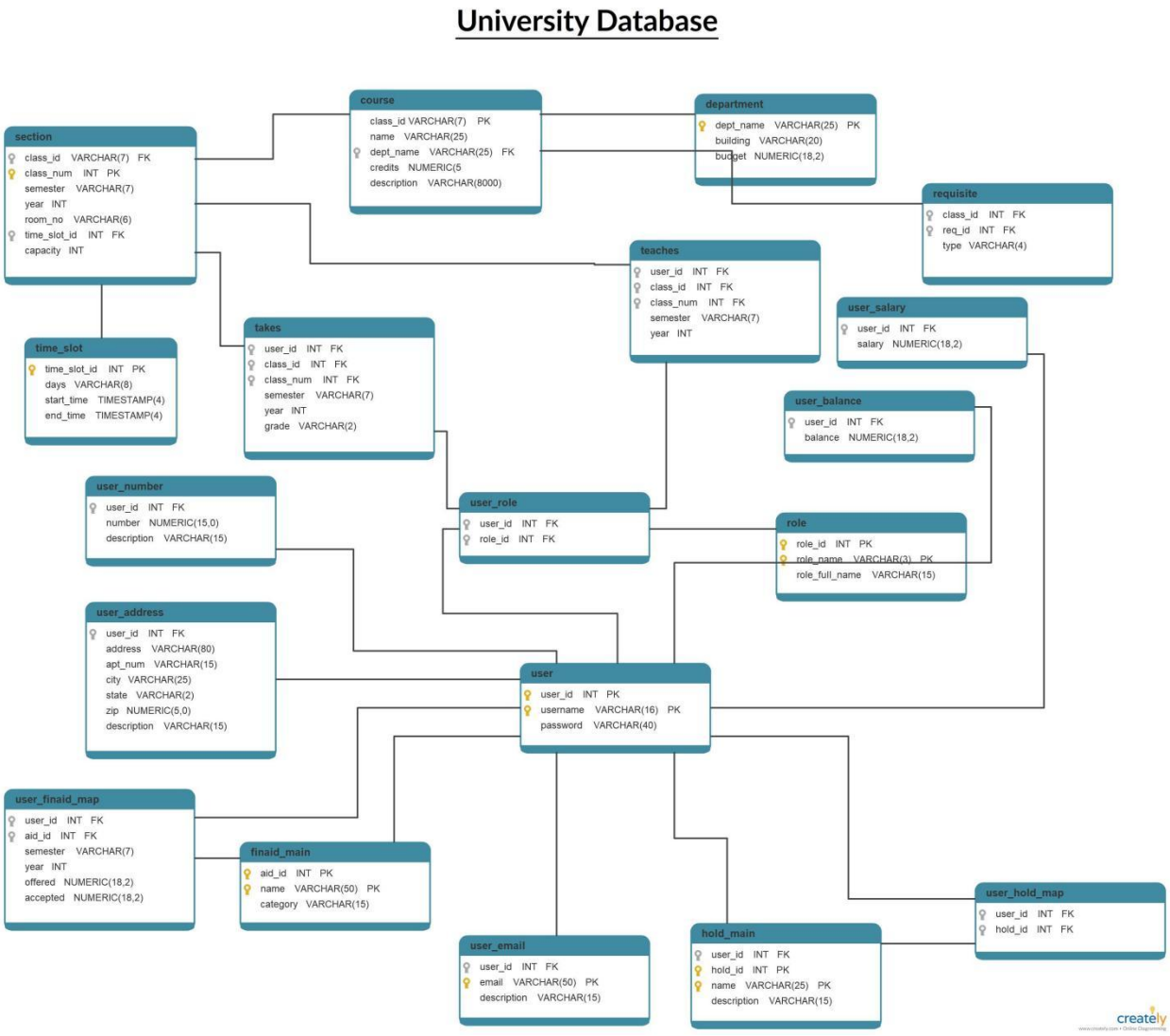


Figure 4.8: Entity-Relationship Diagram of the OBU University Website and Administrative CM

4. 7.1 Database Technology

PostgreSQL is used to store the system's main data. The database supports different areas of the website, including academic information, university content, news, announcements, events, research, services, vacancies, users, and media.

4. 7.2 Main Data Categories

The main types of information stored include:

Academic units, departments, and programs

University and administrative information

News and announcements

Events and galleries

Research and community services

Vacancies and career information

Contact messages

User and CMS information

Media-related data

4. 7.3 Database and Application Relationship

The application communicates with PostgreSQL through the server-side functions.

User Interface → Application/Server → PostgreSQL Database

When administrators add or update content, the application sends the changes to the database. Public pages can then retrieve and display the stored information. This makes the website more dynamic and easier to maintain.

4. 7.4 Data Management

The CMS supports common database operations such as creating, viewing, updating, and deleting content. Administrators can also publish or unpublish content where required.

4. 7.5 Database Security

Database access is supported by PostgreSQL Row-Level Security (RLS). Together with Supabase Authentication and authorization, this provides additional protection for stored data and administrative operations.

4. 7.6 Database Verification

Database operations were included in system testing. Verification covered database communication, authentication, media operations, error handling, and other application functions.

4. 8 Data Flow and Process Design

Data flow describes how information moves between users, the website, application functions, authentication services, and the database. The Oda Bultum University system uses a database-

driven approach where public users mainly view information and administrators manage content through the CMS.

4. 8.1 General Data Flow

The main data flow is:

User → Web Browser → Frontend → Application/Server → Database/Storage → Frontend → User

The application processes requests and connects the website with stored information and supporting services.

4. 8.2 Public User Flow

Public users can access university information without entering the CMS. The basic process is:

User → Website → Application → Database/Storage → Website → User

This supports the display of academic information, news, announcements, events, services, vacancies, research, and other institutional content.

4. 8.3 Administrative Flow

Administrators use a protected process:

Administrator → Login → Authentication → Authorization → CMS → Application → Database/Storage

After successful authentication, authorized administrators can manage website content through the CMS.

4. 8.4 Content Management Flow

When an administrator adds or changes information, the CMS sends the request to the application layer, which stores the changes in the database.

CMS → Application/Server → Database → Updated Content

The updated information can then be displayed on the appropriate website pages.

4. 8.5 Content Retrieval

For database-driven pages, information follows this process:

Page Request → Application → Database → Returned Data → Frontend → Display

This allows website content to be updated through the CMS without changing the source code.

4. 8.6 Authentication and Media Flow

Authentication uses Supabase Authentication and sessions to protect administrative functions. Media uploaded through the CMS is handled through the application and storage services and can then be displayed on the website.

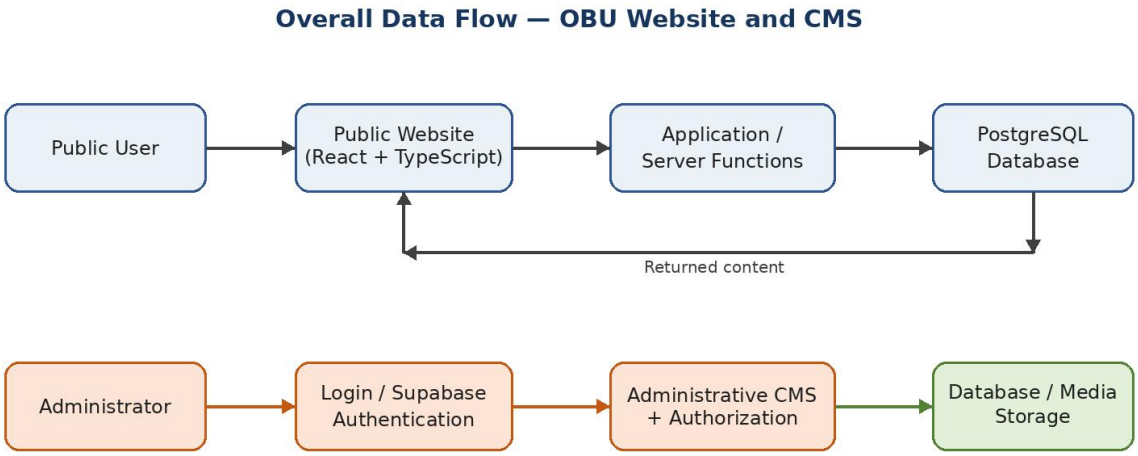
Admin → CMS → Authentication/Authorization → Application → Database/Storage

4. 8.7 Security and Verification

PostgreSQL Row-Level Security (RLS) provides an additional database security layer. The system was also tested for authentication, database operations, media handling, errors, and communication between the application and database.

4. 8.8 Overall Data Flow Diagram

Figure 4.9: Overall System Data Flow Diagram



4. 9 System Design Summary

The system design of the Oda Bultum University website and CMS defined how the main parts of the system work together. It covered the website interface, CMS, application layer, database, security, deployment environment, and data flow.

The system has two main parts: a public website for accessing university information and a protected CMS for authorized administrators. The main users of the system are students, prospective students, university staff, researchers, job seekers, the general public, and administrators.

The main design areas included:

System architecture – shows how the website, CMS, application, database, and services are connected.

User interface – defines how visitors and administrators interact with the system.

Database design – uses PostgreSQL and Supabase to store and manage system information.

Security – uses authentication, authorization, sessions, and Row-Level Security.

Deployment – covers the development, testing, storage, and version-control environment.

Data flow – explains how information moves between users, the application, and database.

Testing – checks system functions, database operations, authentication, and other components.

A security review also identified a server-side authentication issue that requires further correction.

Overall, the design provides the foundation for developing a database-driven university website and CMS. The next chapter discusses the implementation of the designed system, including the technologies, development process, challenges, and testing.

CHAPTER FIVE

SYSTEM IMPLEMENTATION

5. 1 Introduction

This chapter explains how the Oda Bultum University website and CMS were developed from the system design. It covers the tools used, implementation process, challenges, and testing. The system was developed using React, TypeScript, TanStack Start/Router, Tailwind CSS, Node.js, PostgreSQL, and Supabase.

5. 2 Development Environment and Tools

The main tools used during development were Visual Studio Code, Node.js, Git, GitHub, Vite, Playwright, and web browsers. The main technologies included TypeScript, React, TanStack Start/Router, Tailwind CSS, PostgreSQL, and Supabase.

5. 3 Frontend Implementation

The frontend included the public university website and the administrative CMS. The public website provided pages for academic information, departments, programs, leadership, research, services, vacancies, news, events, galleries, and contact. The CMS allowed authorized users to manage university content.

5. 4 Backend and Database Implementation

The application layer handled communication between the frontend, database, authentication, and storage services. PostgreSQL was used for data storage, while Supabase provided database and related services.

5. 5 Authentication and Security

Supabase Authentication was used for login and session management. Authorization protected administrative functions, while PostgreSQL Row-Level Security provided an additional database security layer.

5. 6 Media and Content Management

The CMS supported media management and allowed administrators to create, update, and manage university content. This database-driven approach reduced the need to change source code whenever website information was updated.

5. 7 Routing and Responsive Interface

The system included navigation between public and administrative pages. Protected CMS routes were separated from public pages. The interface was also designed to support different screen sizes using Tailwind CSS.

5. 8 Implementation Challenges and Solutions

The main challenges included database integration, authentication, media management, and maintaining consistency between the frontend, backend, and database. These problems were addressed through testing, correction, and repeated verification.

5. 9 Security Findings

Security was included throughout development. However, the review found that some server-side functions did not properly verify Supabase authentication before accessing PostgreSQL. This was identified as an issue requiring correction.

5. 10 Testing

Testing was performed during development and covered the public website, CMS, navigation, database, authentication, media, responsive behavior, errors, and application/database interactions.

The main process was:

Implement → Test → Verify → Correct → Retest → Integrate

Table 5.1: System Testing Record

Test area	Expected result	Actual result	Status	Evidence/remarks
Public pages and navigation	Pages open and links navigate correctly.	Homepage and other pages opened successfully.	Pass	Pages were checked successfully.
Administrator login	Valid credentials allow authorized access.	Administrator logged in and accessed the dashboard.	Pass	Successful login observed.
Database connection	Database-driven information loads correctly.	Database-driven information loaded successfully.	Pass	Database connection test passed.
Invalid administrator login	Incorrect credentials are rejected.	Incorrect login credentials were rejected.	Pass	Unauthorized login was denied.
Authorization and protected routes	Unauthorized users cannot access protected actions.	Unauthorized access to protected pages was blocked.	Pass	Access-control test passed.
Content management	Administrators can create and update website content.	Website content was created and updated successfully.	Pass	Content-management test passed.
Responsive layout	Pages remain usable on desktop and mobile screens.	Website layout displayed correctly on desktop and mobile.	Pass	Responsive design test passed.

5. 11 Implementation Results

The implementation produced a university website and CMS with major features such as academic information, leadership, research, services, vacancies, news, events, galleries, media, contact functions, user management, and protected administration.

CHAPTER 6

INTERNSHIP REFLECTION AND CONCLUSION

6. 1 Learning Outcomes

During the internship, I gained practical experience in web application development and learned how system design is changed into a working application. I improved my skills in React, TypeScript, Tailwind CSS, database integration, authentication, routing, and testing.

I also learned how to work with Git and GitHub, solve technical problems, test system functions, and connect frontend, backend, and database components. The project helped me better understand how theoretical knowledge is applied in a real development environment.

6. 2 Challenges Faced

Several challenges were encountered during the project. These included database integration, authentication and protected routes, media management, and maintaining consistency between the frontend, backend, and database.

Some security issues were also identified during testing, particularly the need for proper authentication checks in server-side functions. These challenges were addressed through testing, debugging, correction, and repeated verification.

6. 3 Recommendations

Based on the internship experience, the following recommendations are suggested:

Improve server-side authentication and authorization checks.

Continue regular testing of database and security functions.

Keep project documentation updated during development.

Provide more practical training on modern web development tools.

Encourage regular communication between students, supervisors, and development teams.

Continue improving the CMS based on future university requirements.

6. 4 Conclusion

The internship provided valuable practical experience in developing a university website and CMS. The project helped me improve both my technical and problem-solving skills and understand the complete development process from requirements and design to implementation and testing.

Overall, the experience connected academic knowledge with practical software development and provided useful skills for future work in web development.

APPENDICES

APPENDIX A: PROJECT SOURCE CODE

This appendix contains selected source code from the OBU website and Content Management System. The code demonstrates how the application connects to the database and manages website functionality.

A.1 Database Connection Project file:
src/backend/database/db.ts

```
/**
 * PostgreSQL database layer for OBU.
 * Uses pg (node-postgres).
 */
import { Pool } from "pg";
import "dotenv/config";
let _pool: Pool | null = null;
let _initialized = false;

export function getDb(): Pool {
  if (_pool) return _pool;

  const connectionString = process.env["DATABASE_URL"];

  if (!connectionString) {
    throw new Error("DATABASE_URL environment variable is missing.");
  }

  _pool = new Pool({
    connectionString,
  });

  return _pool;
}

export async function initDb() {
  if (_initialized) return;
  const pool = getDb();
  await initSchema(pool);
  _initialized = true;
}
```

A.2 Database Type Definitions

Project file: src/frontend/lib/database.types.ts



A.3 Database Migration Files

Project location: `supabase/migrations/`

These SQL files document database changes made during project development.

```sql
-- 1. Add slug columns
ALTER TABLE public.galleries ADD COLUMN IF NOT EXISTS slug TEXT;
ALTER TABLE public.leadership ADD COLUMN IF NOT EXISTS slug TEXT;
ALTER TABLE public.services ADD COLUMN IF NOT EXISTS slug TEXT;
ALTER TABLE public.academic_units ADD COLUMN IF NOT EXISTS slug TEXT;
ALTER TABLE public.programs ADD COLUMN IF NOT EXISTS slug TEXT;
ALTER TABLE public.administrative_offices ADD COLUMN IF NOT EXISTS slug TEXT;

-- 2. Create slugification helper function
CREATE OR REPLACE FUNCTION public.generate_slug(base_text TEXT)
RETURNS TEXT AS $$
DECLARE
    base_slug TEXT;
BEGIN
    -- Lowercase, replace non-alphanumeric with hyphens, trim hyphens
    base_slug := lower(regexp_replace(base_text, '[^a-zA-Z0-9]+', '-', 'g'));
    base_slug := trim(both '-' from base_slug);

    IF base_slug IS NULL OR base_slug = '' THEN
        base_slug := 'item';
    END IF;

    RETURN base_slug;
END;
$$ LANGUAGE plpgsql;

-- 3. Backfill existing data
-- galleries (title)
```

REFERENCES

The references below are presented in IEEE style. Please verify each source and complete any missing publication details before submission.

1. Oda Bultum University, “About Oda Bultum University,” OBU, Chiro, Ethiopia, 2026. [Online]. Available: https://www.obu.edu.et

2. Haramaya University, “Industrial Practice Report Guideline (Software Development),” Haramaya University, Ethiopia, 2026.

3. Oda Bultum University, “OBU Website Governance Policy,” OBU, Chiro, Ethiopia, 2025.

4. React, “React Documentation,” Meta, 2026. [Online]. Available: https://react.dev

5. TypeScript, “TypeScript Handbook,” Microsoft, 2026. [Online]. Available: https://www.typescriptlang.org/docs/

6. Tailwind Labs, “Tailwind CSS Documentation,” Tailwind Labs, 2026. [Online]. Available: https://tailwindcss.com/docs

7. Node.js, “Node.js Documentation,” OpenJS Foundation, 2026. [Online]. Available: https://nodejs.org/en/docs

8. Supabase, “Supabase Documentation: Auth, Database, Storage, and Row-Level Security,” Supabase, 2026. [Online]. Available: https://supabase.com/docs

9. GitHub, “GitHub Docs,” GitHub, 2026. [Online]. Available: https://docs.github.com

10. W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

11. IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

12. IEEE, “IEEE Standard for Software and System Test Documentation,” IEEE Std 829-2008, 2008.

13. ISO/IEC, “ISO/IEC 25010:2011 – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models,” ISO, Geneva, Switzerland, 2011.

14.R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed. New York, NY, USA: McGraw-Hill, 2020.

15. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

16. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994.

17. M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

18. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.